

Session 04 – Supervised Learning

Hector J. Hortua, PhD

2026

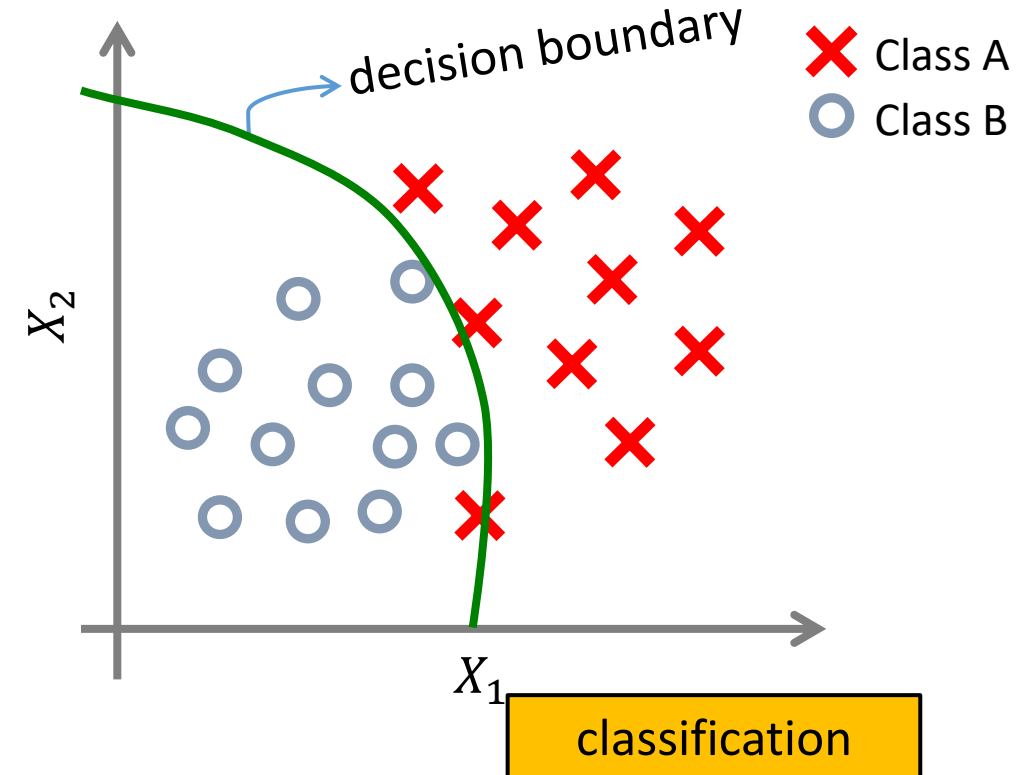
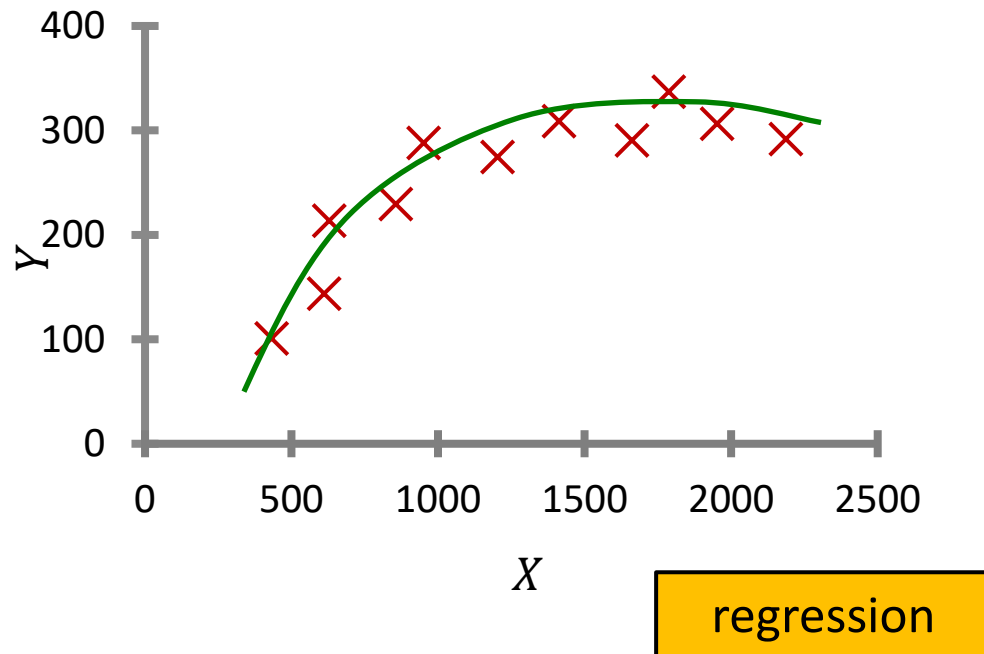
Learning outcomes

By the end of this session we should be able to:

- Describe a range of different machine learning algorithms for supervised learning tasks.
- Implement them in Python using Scikit-Learn.

Supervised Learning

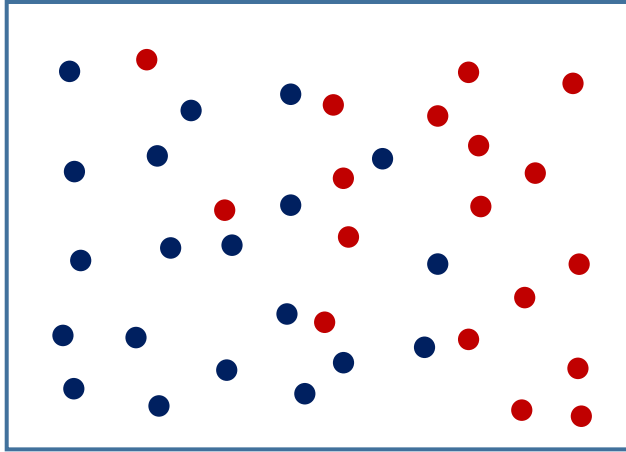
- Input and responses (outputs) are known, which are used to build the model. We want to use the model to predict new responses given new inputs.
- It is related to predictive modelling
- It is useful for regression (continuous target) and classification (categorical target)



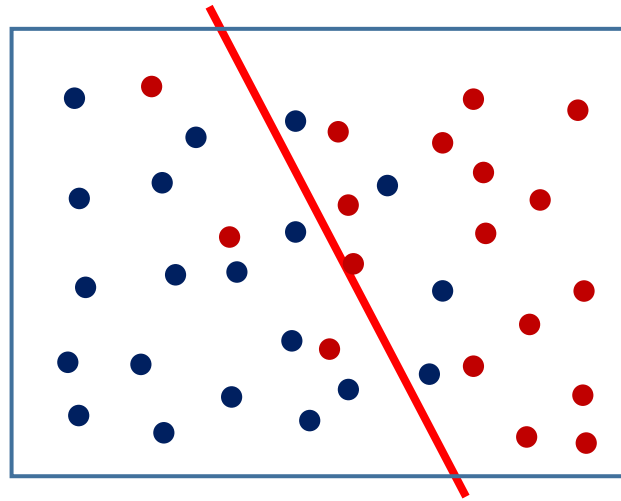
Linear vs Non-linear

Classification

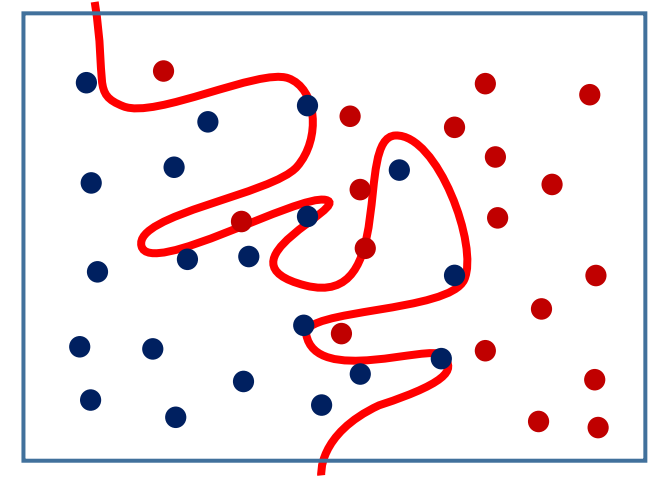
Data



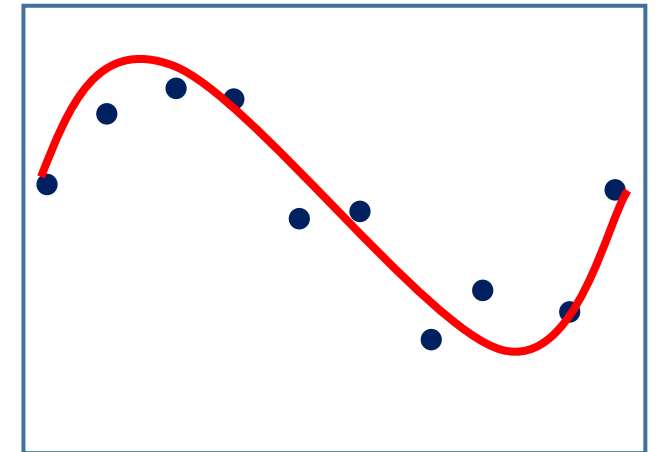
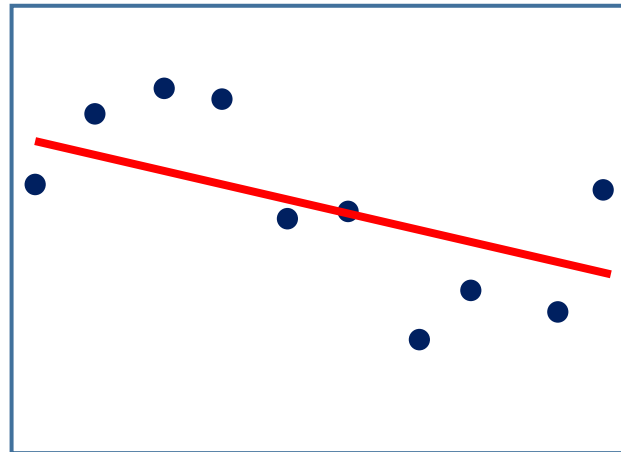
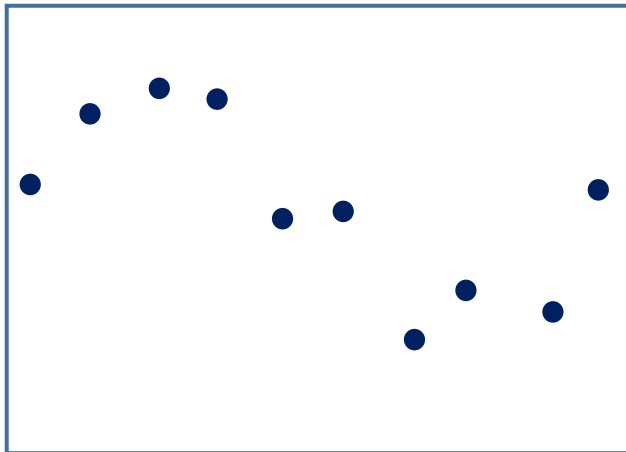
Linear



Non-linear



Regression



Support Vector Machines

Decision boundaries: Hyperplane separation

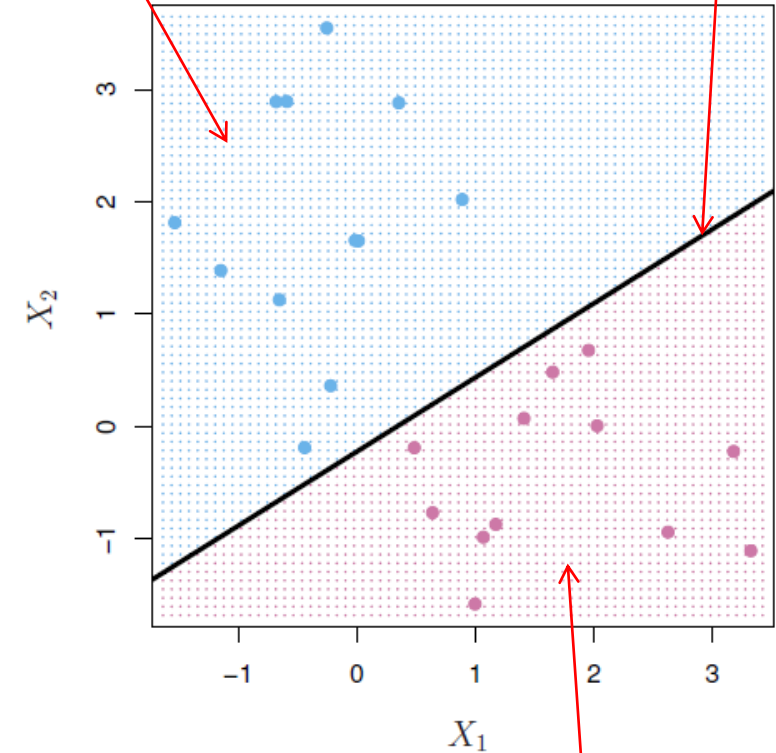
- Let's define the general equation of an hyperplane as:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p = 0$$

- Then, $f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$, can be used to a classification rule as follows:
 - $f(X) > 0$ for points on one side of the hyperplane, and
 - $f(X) < 0$ for points on the other.
 - $f(X) = 0$ defines a **separating hyperplane**.

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 > 0$$

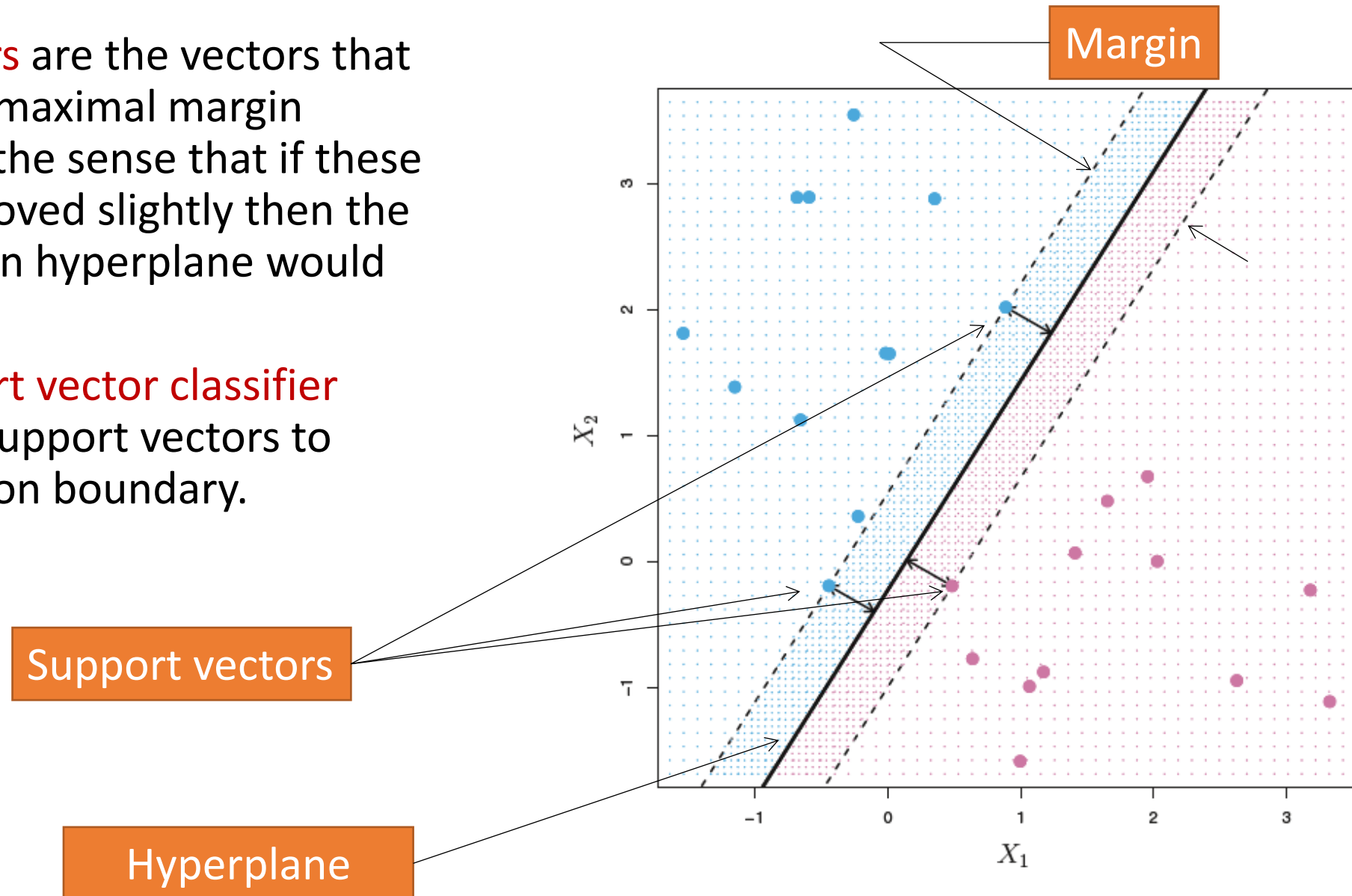
$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$



$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 < 0$$

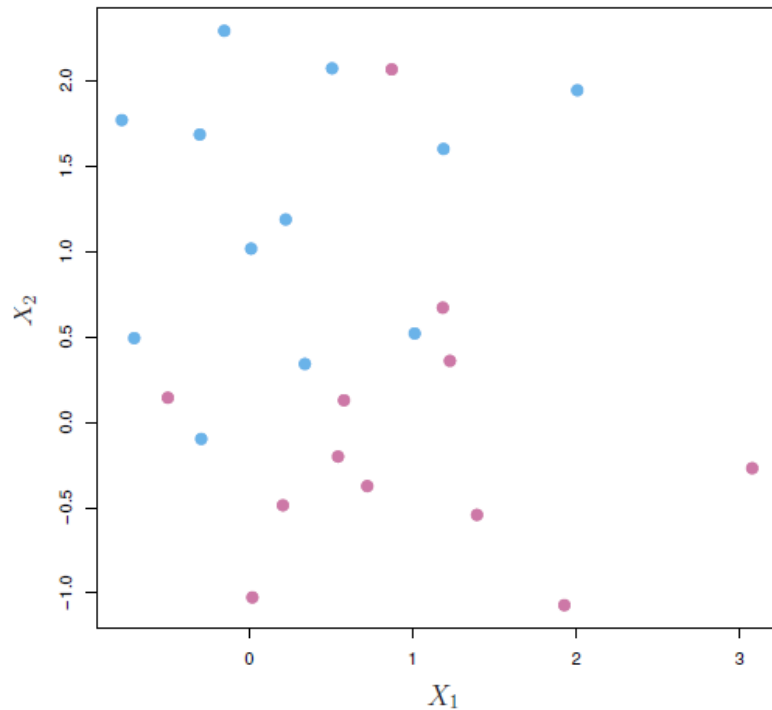
Linear support vector classifier

- **Support vectors** are the vectors that “support” the maximal margin hyperplane in the sense that if these points were moved slightly then the maximal margin hyperplane would move as well.
- A **linear support vector classifier** makes use of support vectors to define a decision boundary.

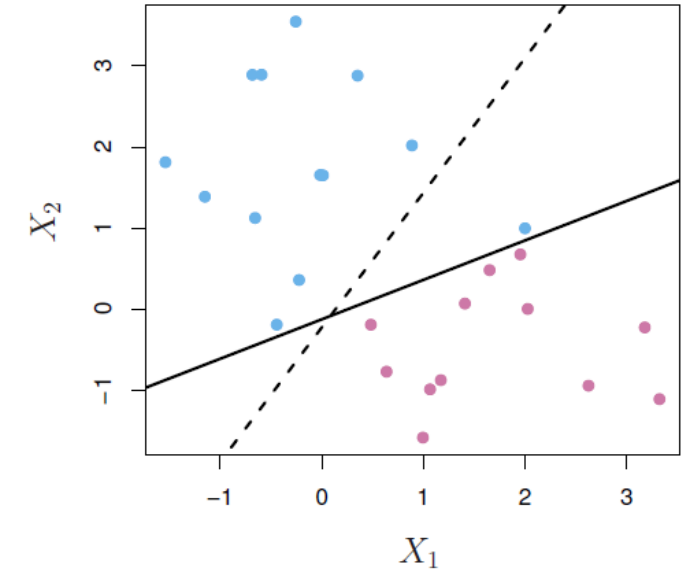
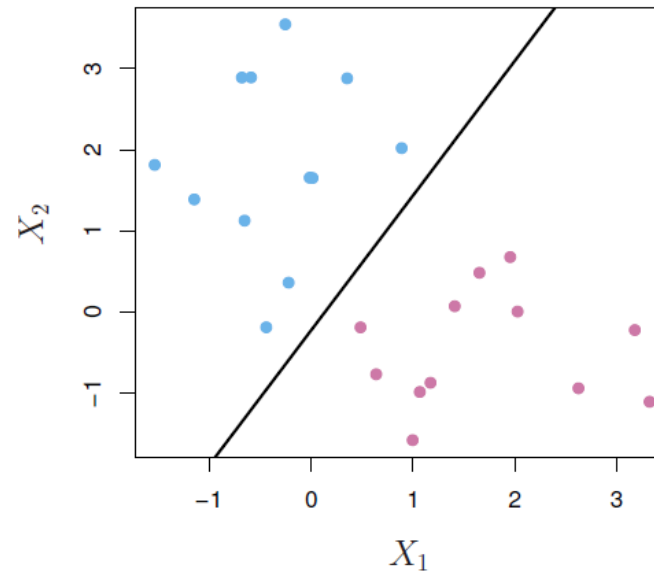


Issues with the linear support vector classifier

- Most of the classification problems require **non-separable** data.



- Sometimes the data are separable, but **noisy**. This can lead to a poor solution for the maximal-margin classifier.



Soft margin classifier

- It is the solution of the optimisation problem:
- Where, C , the **complexity parameter**, is a nonnegative **tuning** parameter, M is the **width** of the margin (that we want to maximise), $\epsilon_1, \dots, \epsilon_n$ are **slack variables** that allow individual observations to be on the wrong side of the margin or the hyperplane.

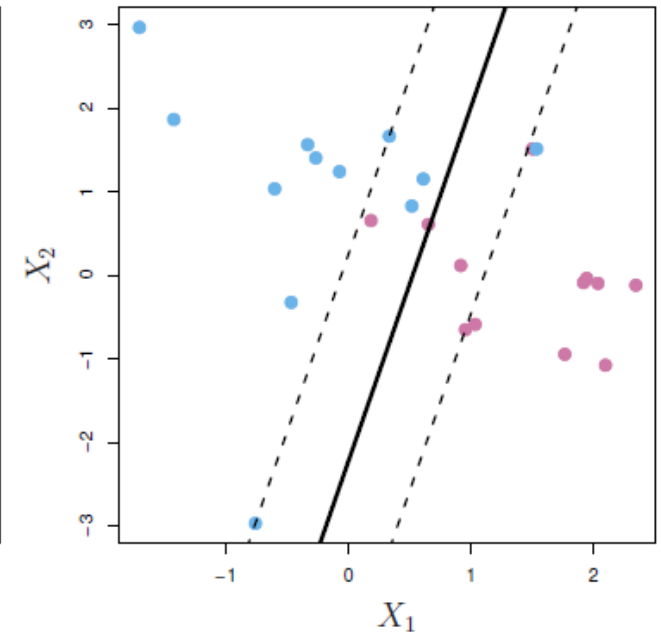
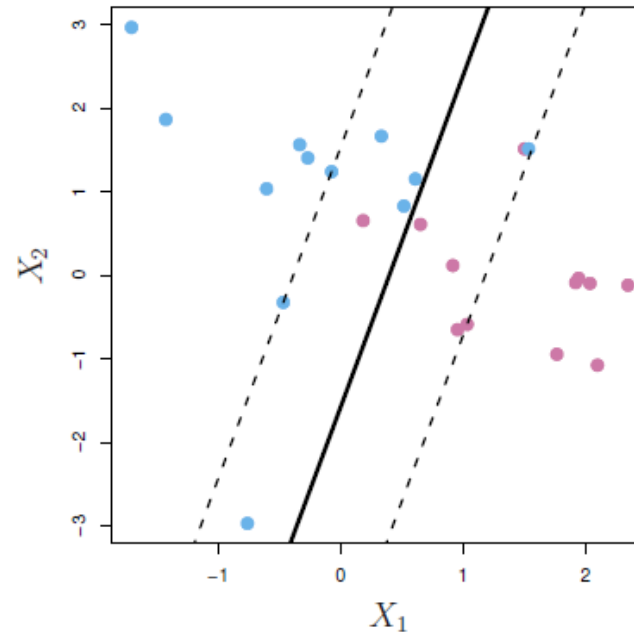
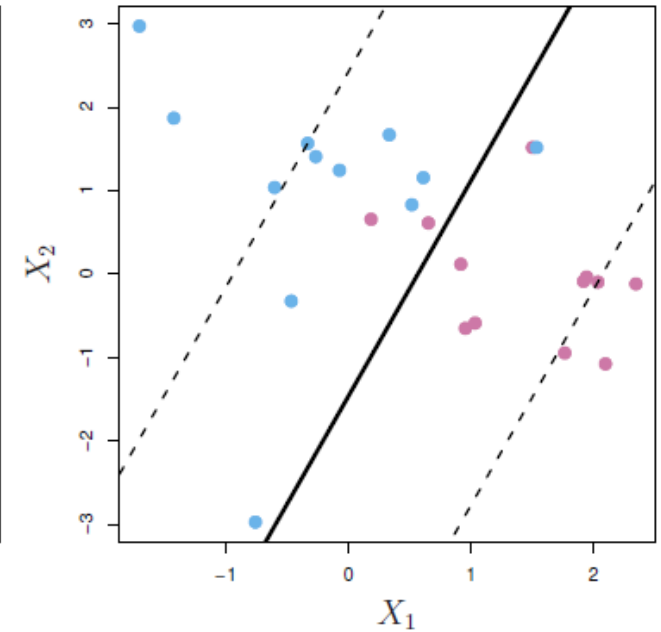
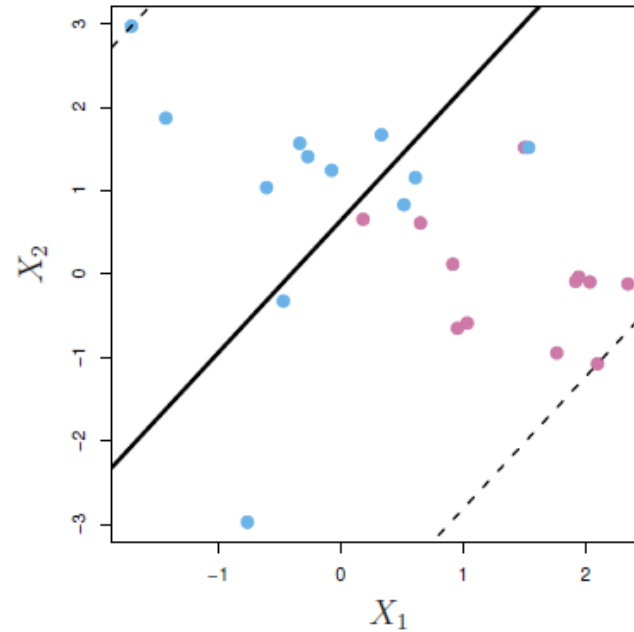
$$\begin{aligned} & \text{maximise } M \\ & \quad \beta_0, \dots, \beta_p \\ & \text{subject to } \sum_{j=1}^p \beta_j^2 = 1, \\ & \quad y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \\ & \quad \epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C \end{aligned}$$

- The **solution** of the soft margin linear classifier (support vector classifier) maximisation problem has the following form:

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle$$

Soft margin classifier

- A **soft margin classifier** allows some observations to be on the incorrect side of the margin, or even the incorrect side of the hyperplane.
- It is controlled by the hyperparameter C .
 - If $C = +\infty$, there is a “**hard**” margin classifier.
 - If $C \rightarrow 0$, it becomes **more tolerant** of violations to the margin.
 - If $C = 0$, there is basically no constraint anymore, and we end up with a hyperplane not classifying anything.



The support vector machine

- From linear algebra, we know that the inner product is a measure of **similarity** between two vectors.
- From previous slide, note that the estimation of α_i parameters does not depend on the data but on the **similarity** across the data observations. It is easy to imagine that the inner product can be **replaced** with a generalisation function for **similarity**: $K(x_i, x_{i'})$.

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i)$$

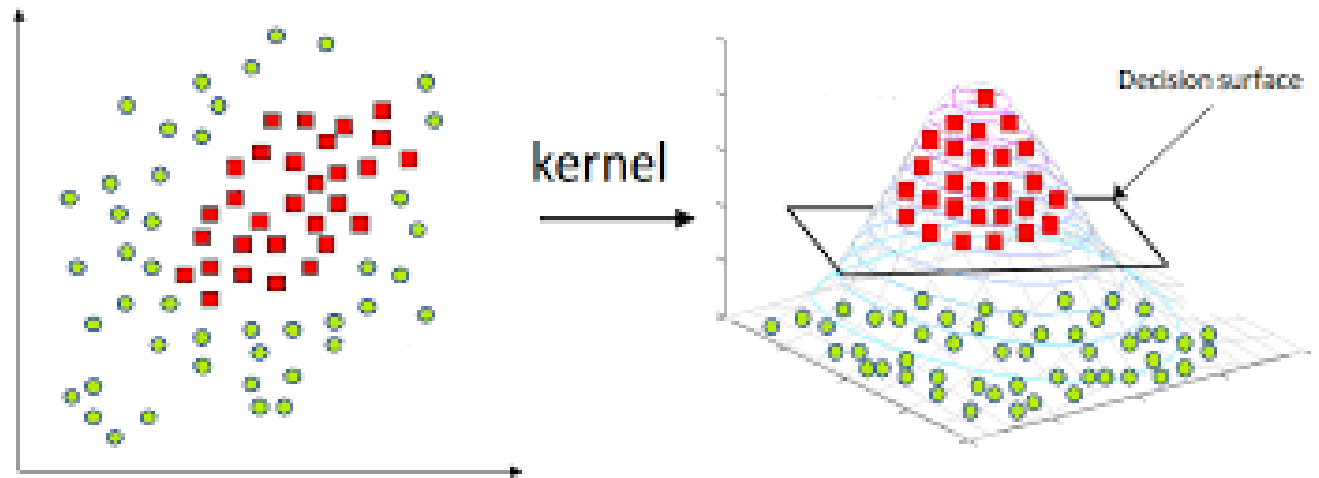
- $K(x_i, x_{i'})$ is called the **kernel function**. In principle, any similarity function can be used provided is symmetric*.
- A **support vector machine** (SVM) is a generalisation of the support vector classifier that uses kernel functions to allow for non-linear separation boundaries.

*A more formal condition is called the Mercer's condition.

Support vector machines (SVM)

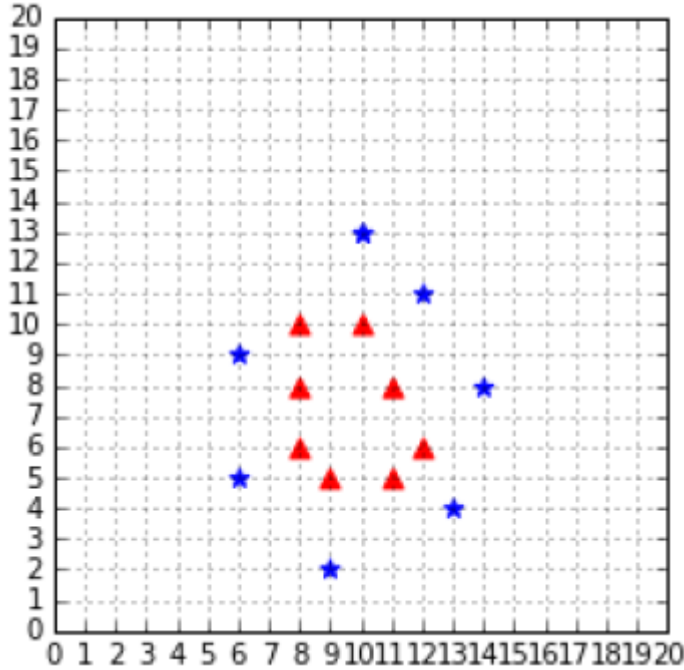
- SVM uses kernel functions to transform the data into a feature space, where it is expected they would be **linearly separable**.
- Formally, a kernel function measures the **similarity** between two data points after they are **transformed** into another, usually **high dimensional** space, named the **feature or Hilbert space**.

$$K(x_i, x_{i'}) = \langle \Phi(x_i), \Phi(x_{i'}) \rangle$$

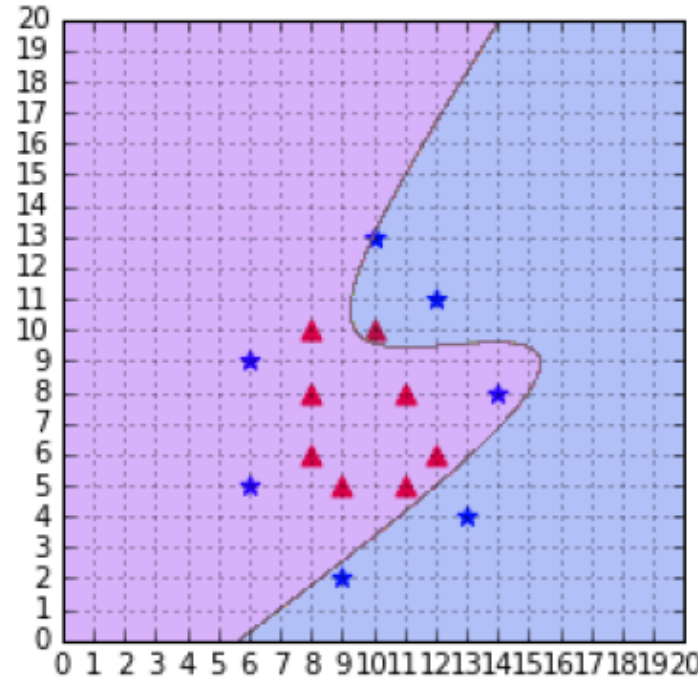


Kernel function examples

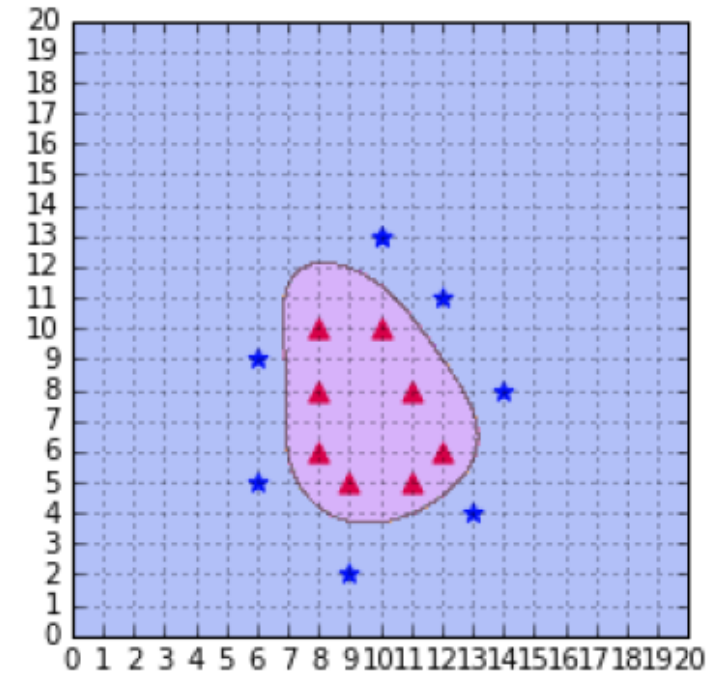
$$K(x_i, x_{i'}) = \exp \left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right)$$



Non-linearly separable problem



A polynomial kernel is not able to separate the data (degree=3, C=100)



A radial basis function (RBF) kernel classifies the data correctly with gamma = 0.1

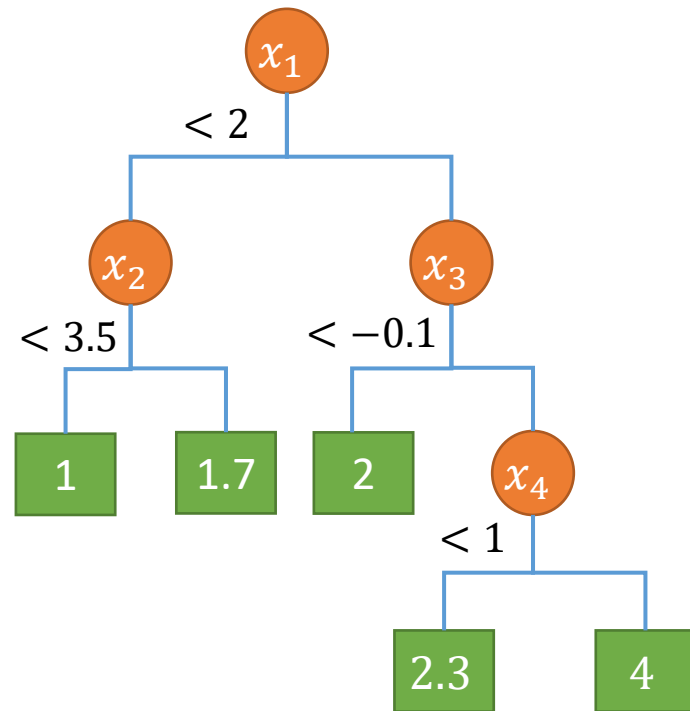
Decision Trees and Ensemble Learning

Classification and Regression Trees (CART)

- Let's assume the following problem:

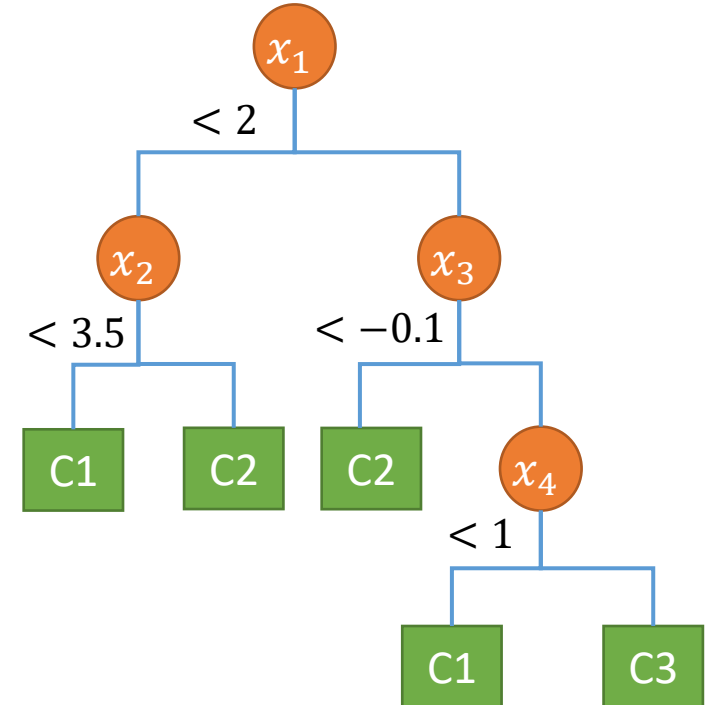
$$Y = f(\mathbf{X})$$

- Decision trees can be used to learn the function f , with Y continuous or categorical, for regression or classification problems, respectively.
- A tree training algorithm will **partition** the space following a diverse of criteria, but CARTs are the most popular.
- Note that in regression trees, the response takes **discrete** values.



IF $x_1 < 2$ & $x_2 < 3.5$ THEN $y = 1.0$
IF $x_1 < 2$ & $x_2 > 3.5$ THEN $y = 1.7$
IF $x_1 > 2$ & $x_3 < -0.1$ THEN $y = 2.0$
IF $x_1 > 2$ & $x_3 > -0.1$ & $x_4 < 1$ THEN $y = 2.3$
IF $x_1 > 2$ & $x_3 > -0.1$ & $x_4 > 1$ THEN $y = 4.0$

Regression tree

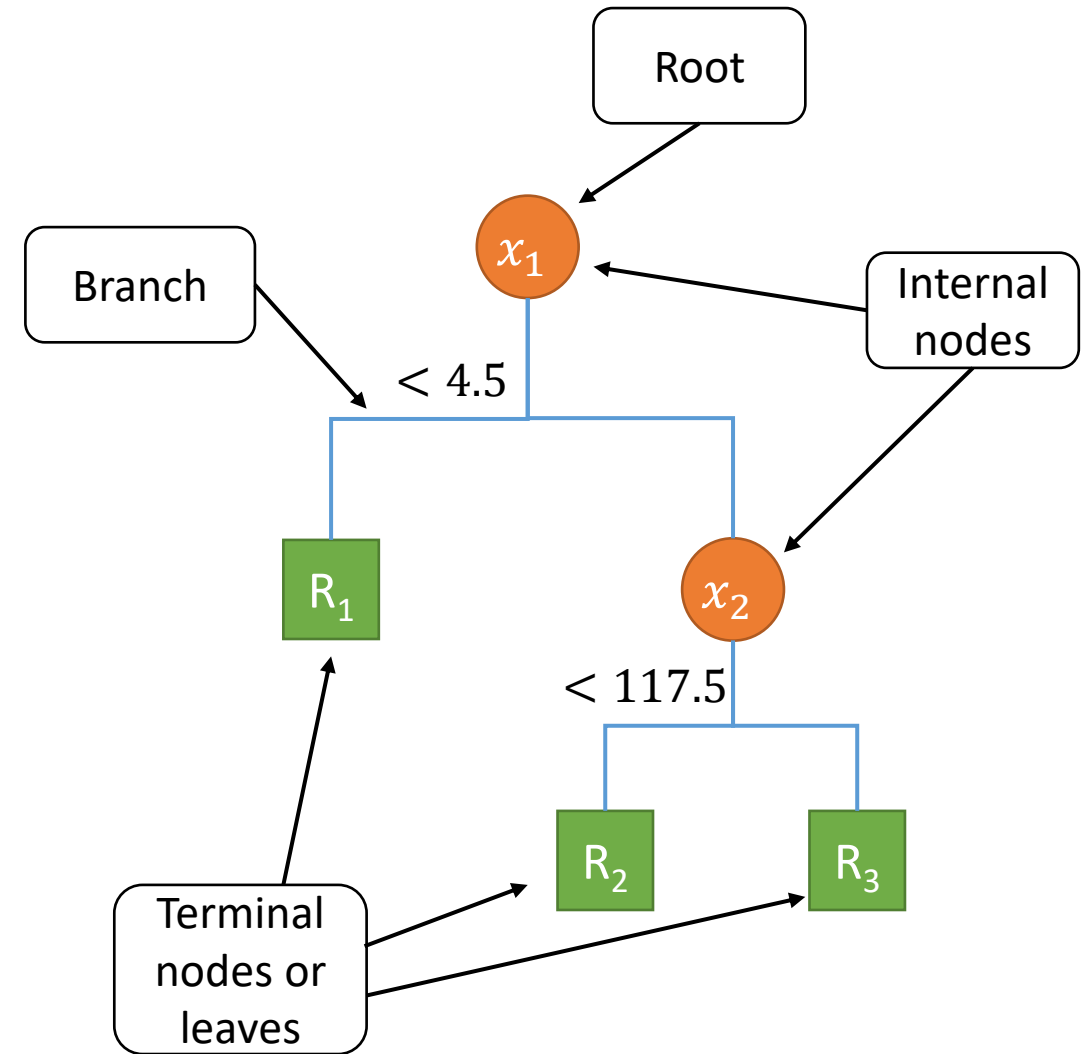


IF $x_1 < 2$ & $x_2 < 3.5$ THEN $y = C1$
IF $x_1 < 2$ & $x_2 > 3.5$ THEN $y = C2$
IF $x_1 > 2$ & $x_3 < -0.1$ THEN $y = C2$
IF $x_1 > 2$ & $x_3 > -0.1$ & $x_4 < 1$ THEN $y = C1$
IF $x_1 > 2$ & $x_3 > -0.1$ & $x_4 > 1$ THEN $y = C3$

Classification tree

Terminology for Trees

- We keep a tree **analogy**.
- Regions R_1 , R_2 , and R_3 are known as **terminal nodes**.
- Decision trees are typically drawn **upside down**, in the sense that the leaves are at the bottom of the tree.
- The points along the tree where the predictor space is split are referred to as **internal nodes**
- Usually, **left-hand** branch corresponds to $X_j < t_k$, and **right-hand** branch, to $X_j \geq t_k$.
- In the hitters tree, the two **internal nodes** are indicated by the text $\text{Years} < 4.5$ and $\text{Hits} < 117.5$.



Loss functions for CARTs

- We can use any error metric for regression trees, but typically the residual sum of squares (RSS) is used.
- For classification, the *misclassification rate* can be used. It is the opposite to the accuracy.

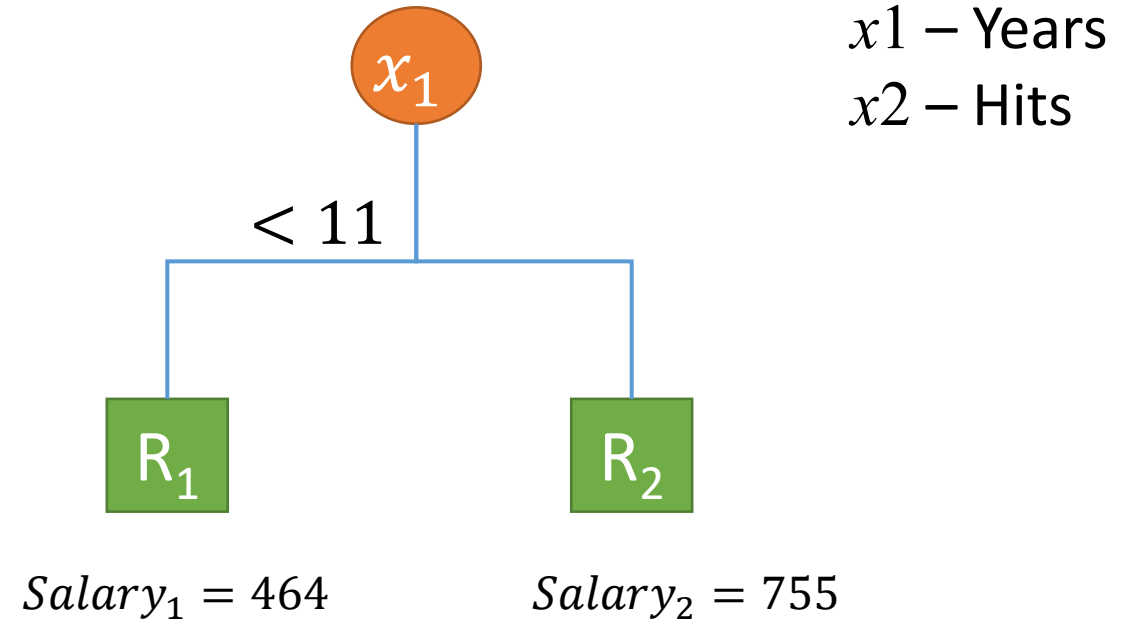
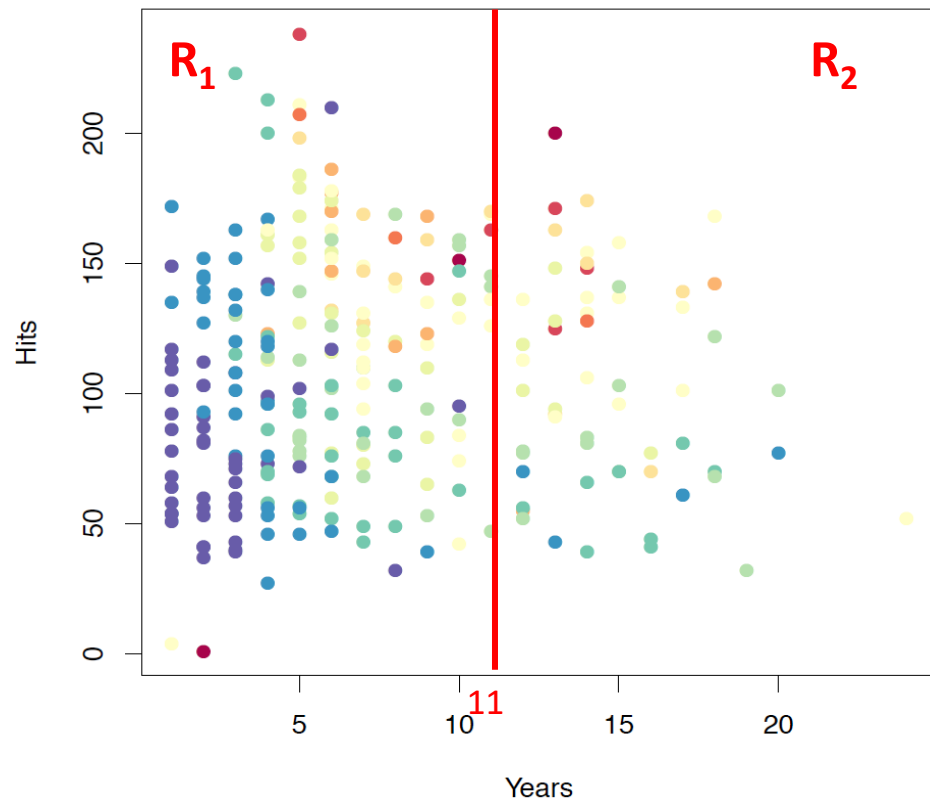
$$E = \sum_{m=1}^M 1 - \max_k(\hat{p}_{mk})$$

- Here $\hat{p}_{mk}$ represents the **proportion** of training observations in the m th region that are from the k th class.
- However, the *Gini index* is preferred as it promotes better data distribution across the tree nodes:

$$G = \sum_{m=1}^M \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}) \quad \text{Entropy} = - \sum_{m=1}^M \sum_{k=1}^K \hat{p}_{mk} \log_2(\hat{p}_{mk})$$

- The Gini index and Entropy measures **node purity** – a small value indicates that a node contains predominantly observations from a single class.

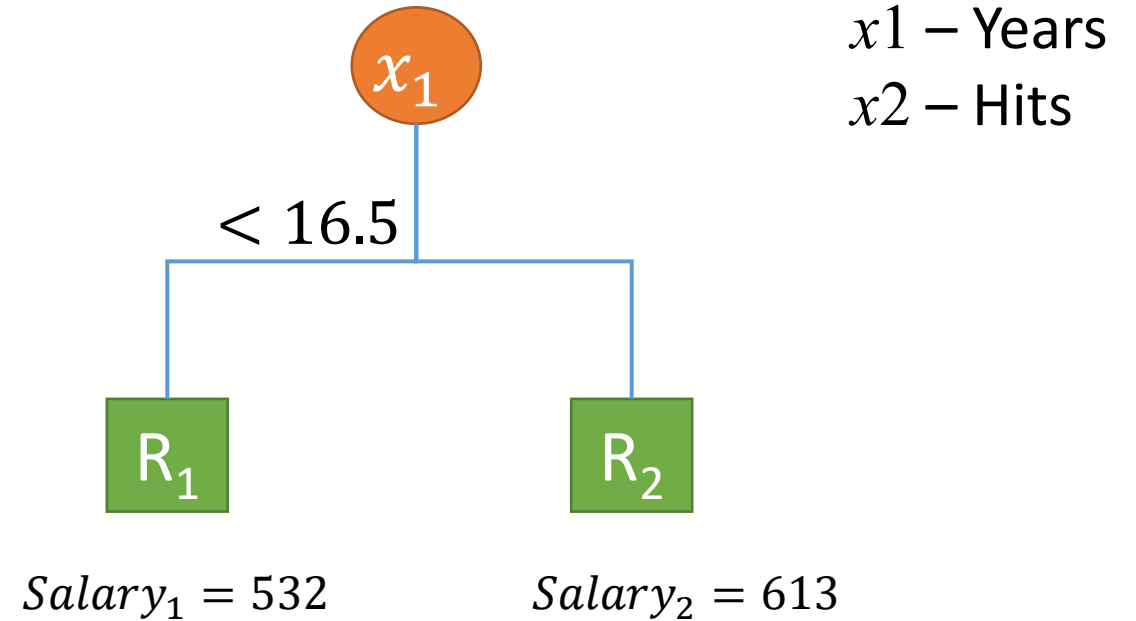
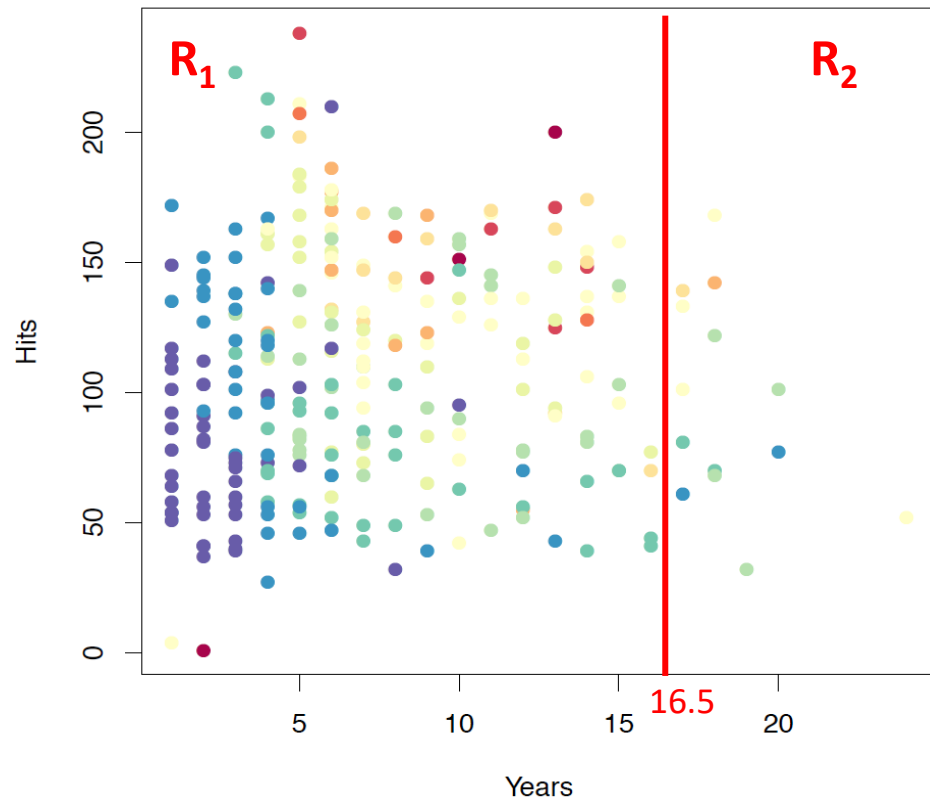
Recursive partitioning example – Hitters data (2 variables)



$RSS = 49171342$

Iteration = 1

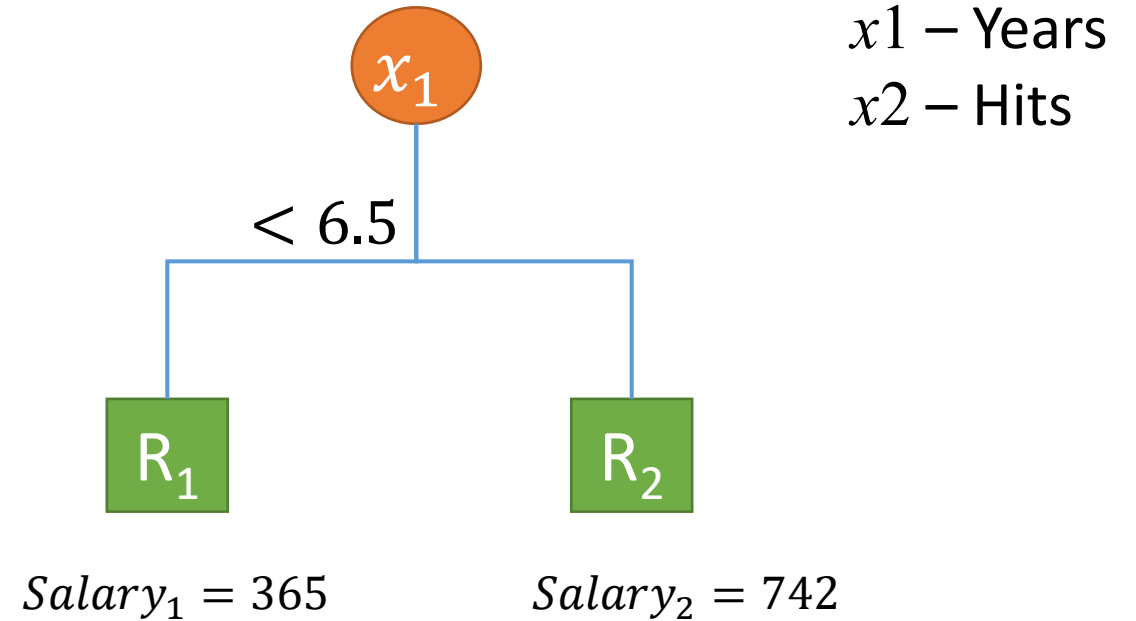
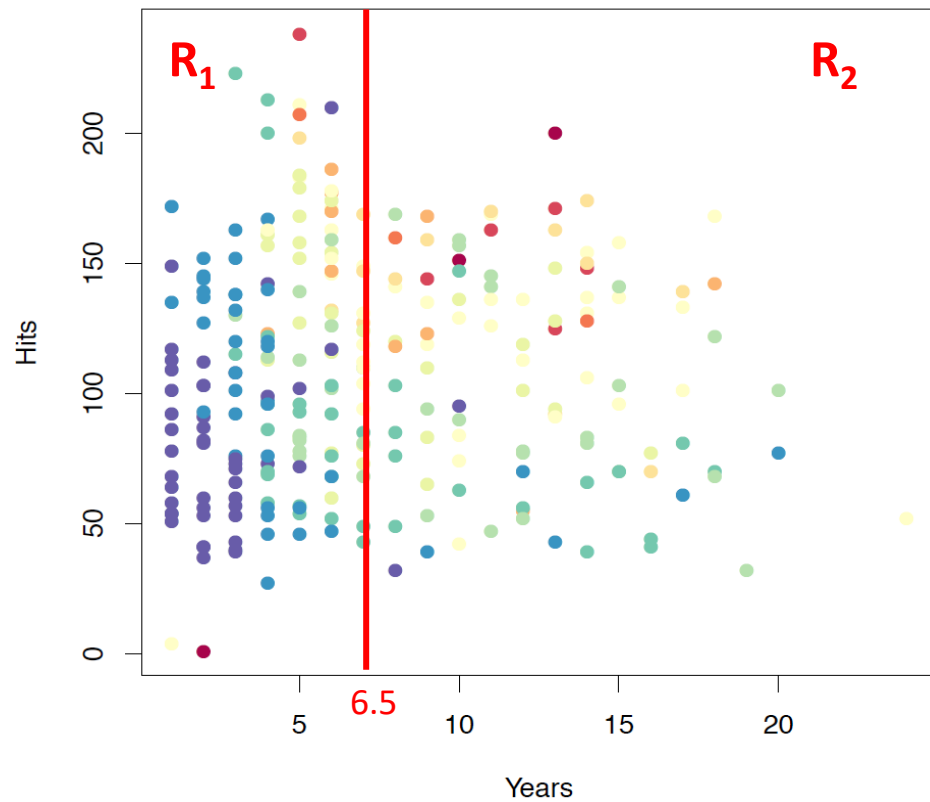
Recursive partitioning example – Hitters data (2 variables)



$RSS = 53232041$ 

Iteration = 2

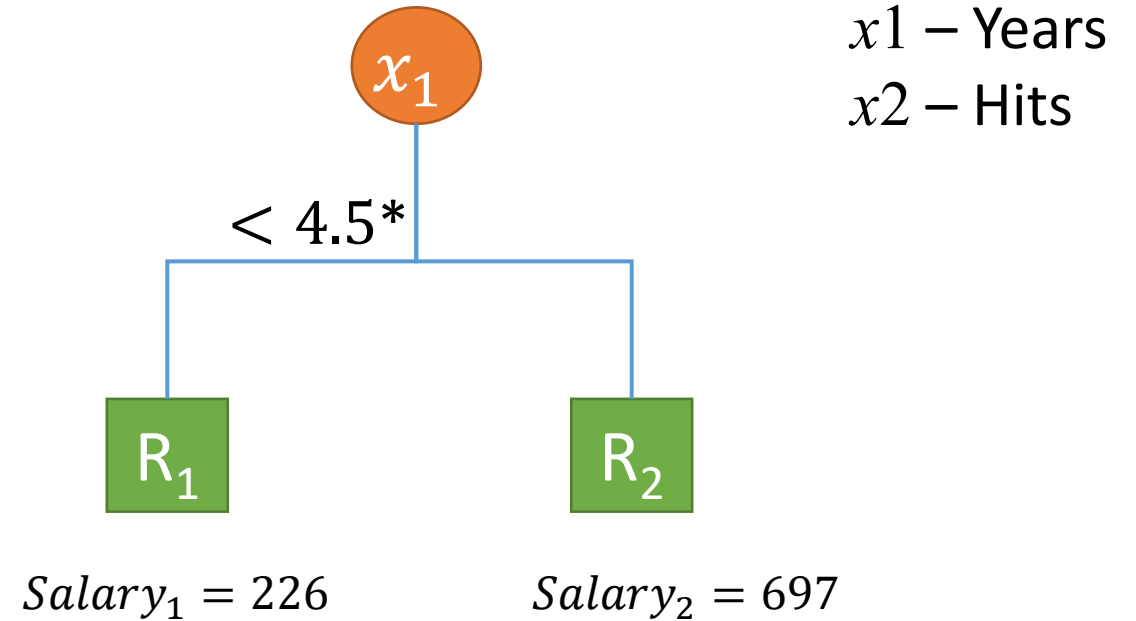
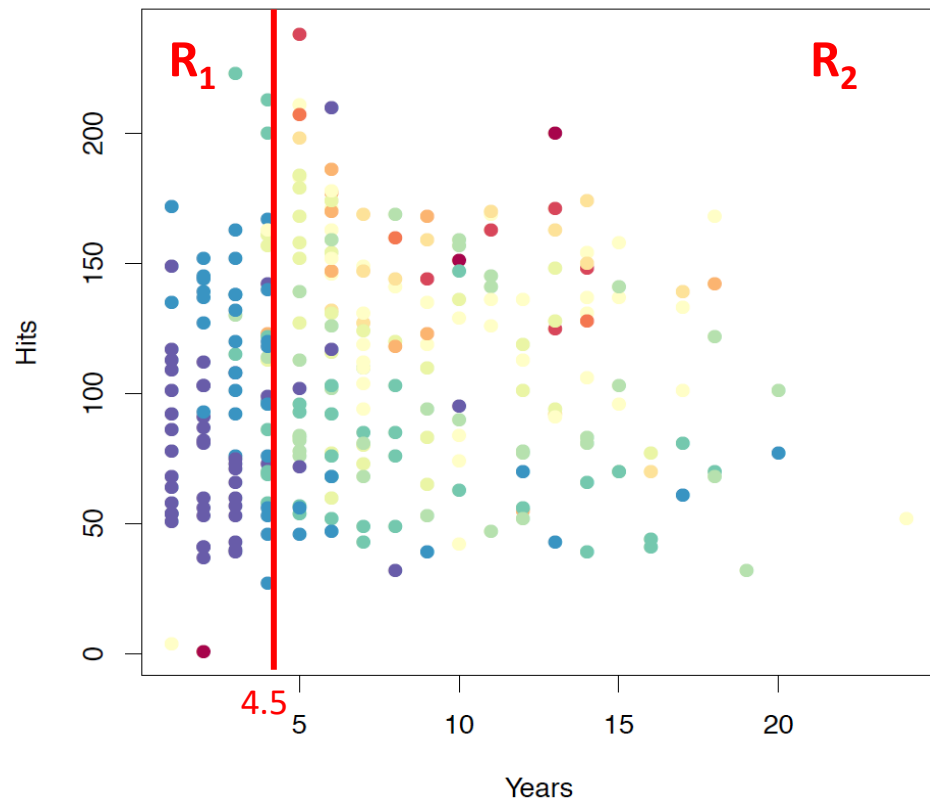
Recursive partitioning example – Hitters data (2 variables)



$RSS = 44051523$ ↓

Iteration = 3

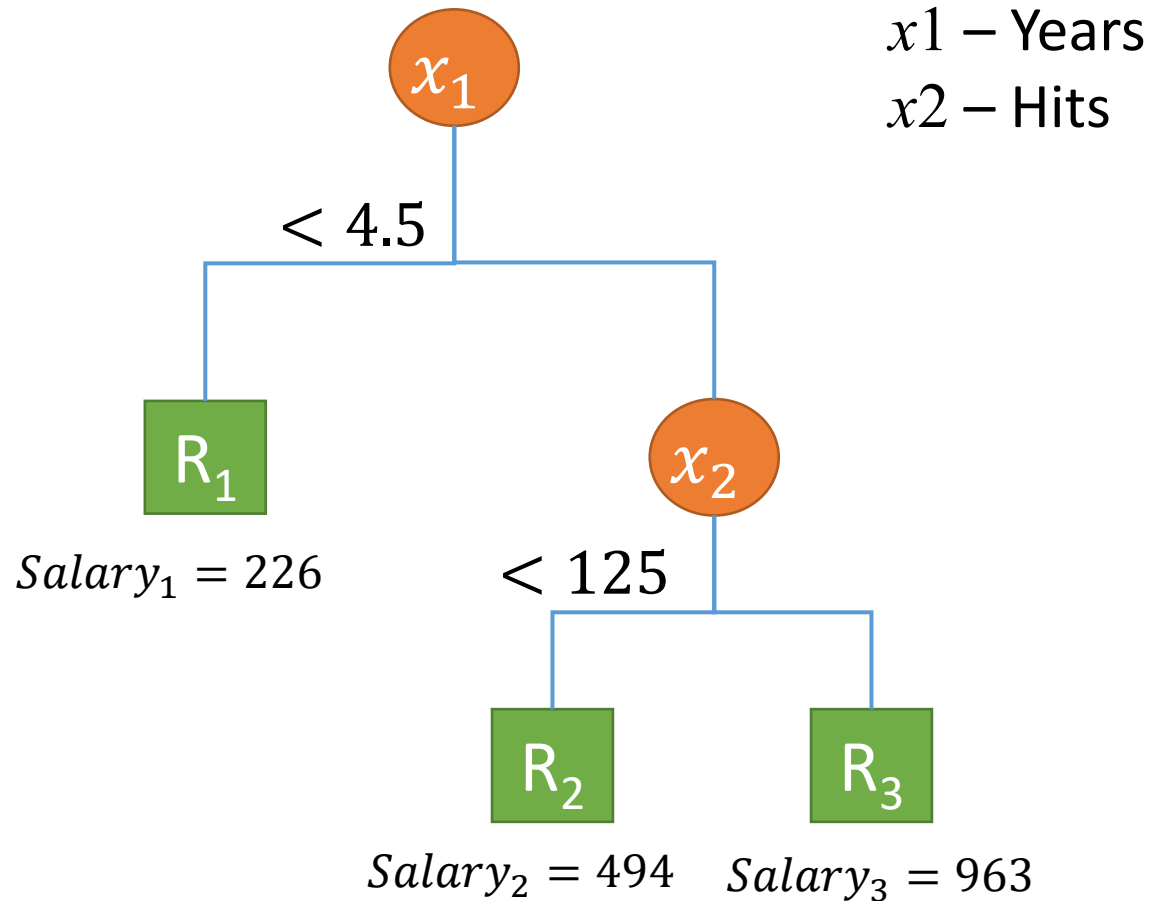
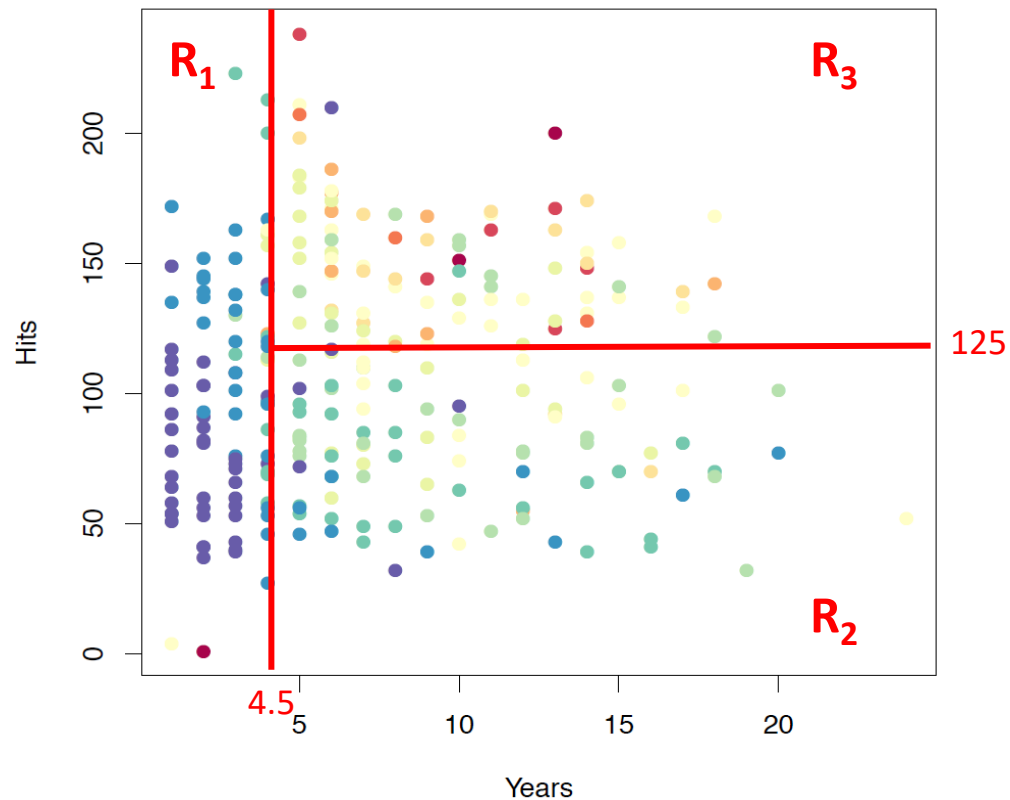
Recursive partitioning example – Hitters data (2 variables)



$RSS = 40162637$ 

*optimum cut-point for *Years*

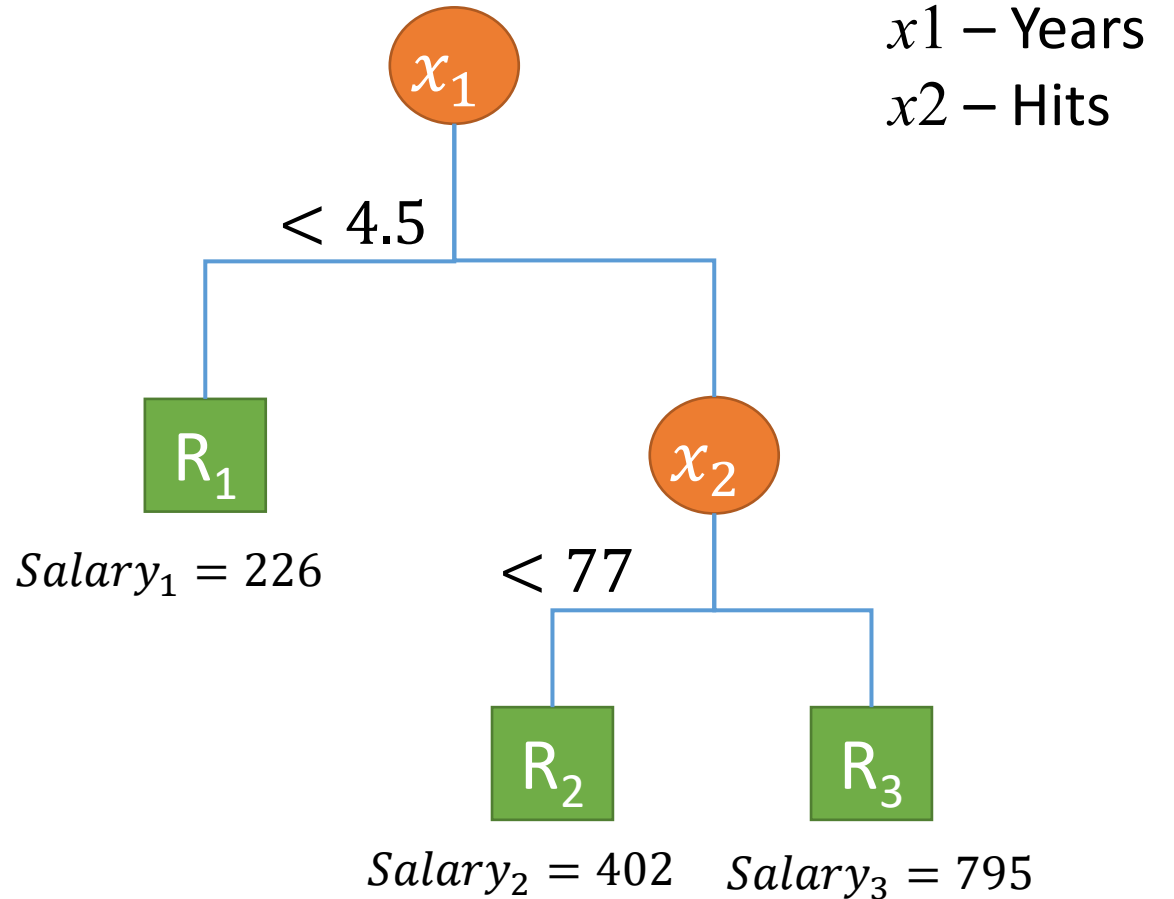
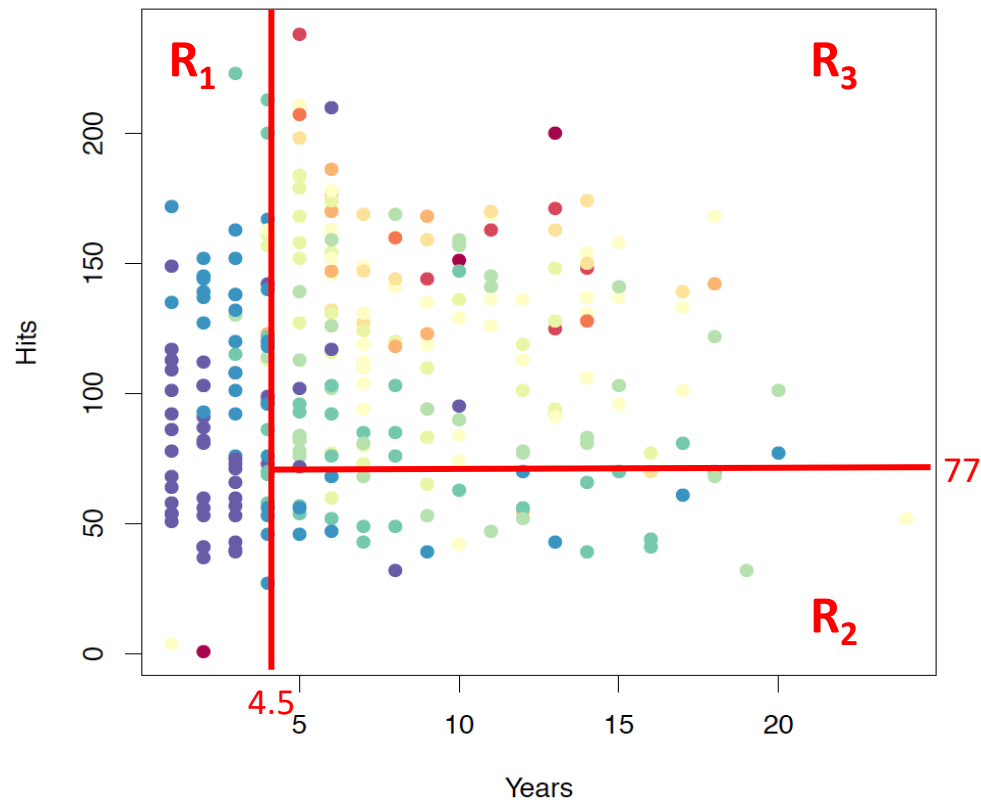
Recursive partitioning example – Hitters data (2 variables)



$RSS = 30820324$ ↓

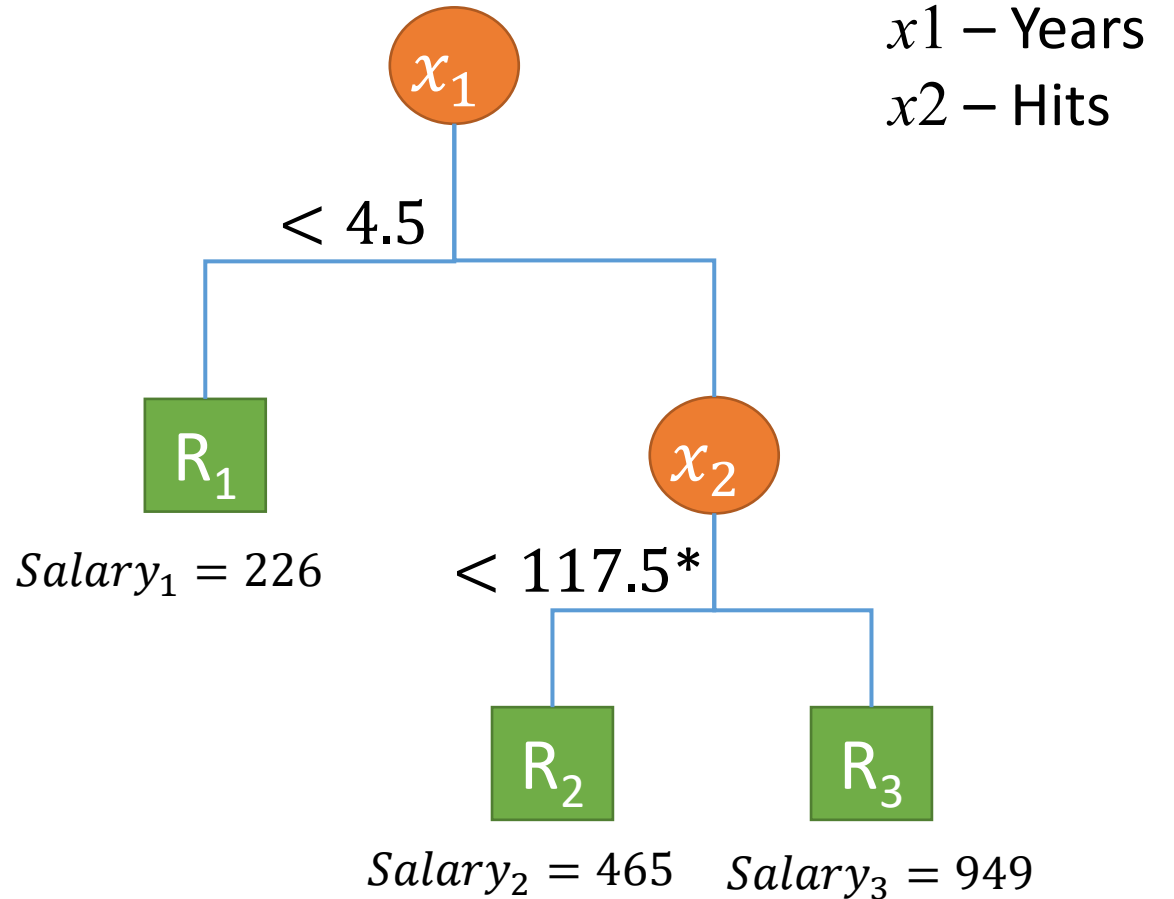
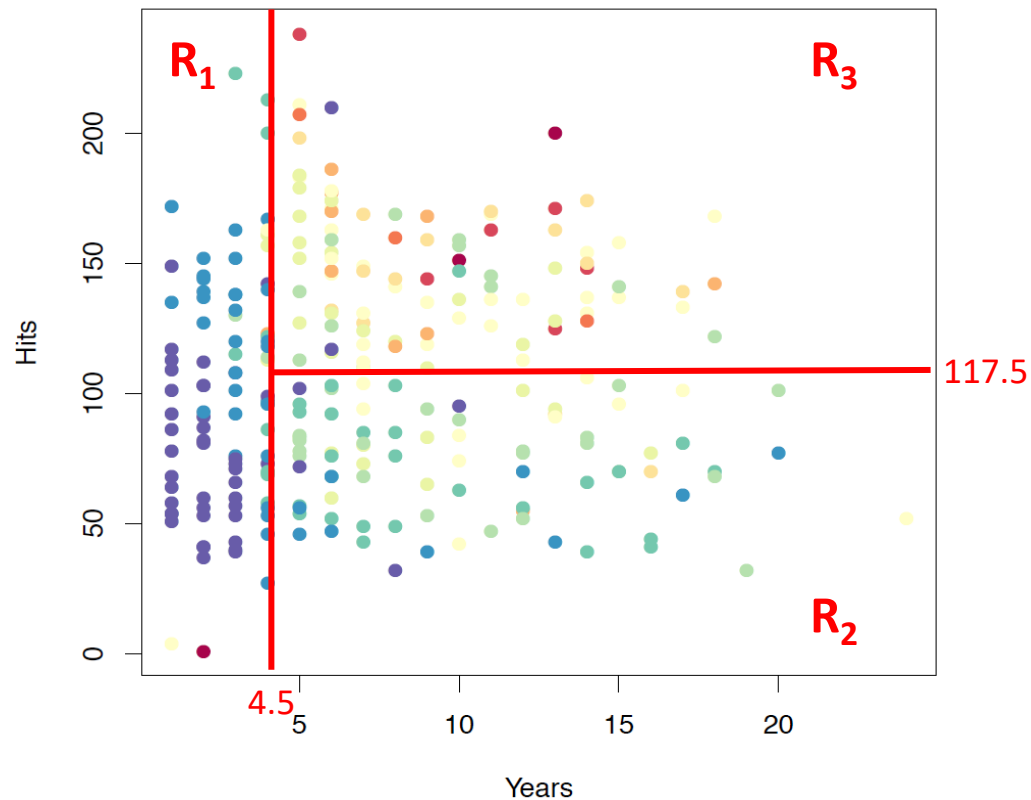
Hits (x_2) variable added, the process is repeated

Recursive partitioning example – Hitters data (2 variables)



$RSS = 35161358$ ↑

Recursive partitioning example – Hitters data (2 variables)



$RSS = 30037022$ ↓

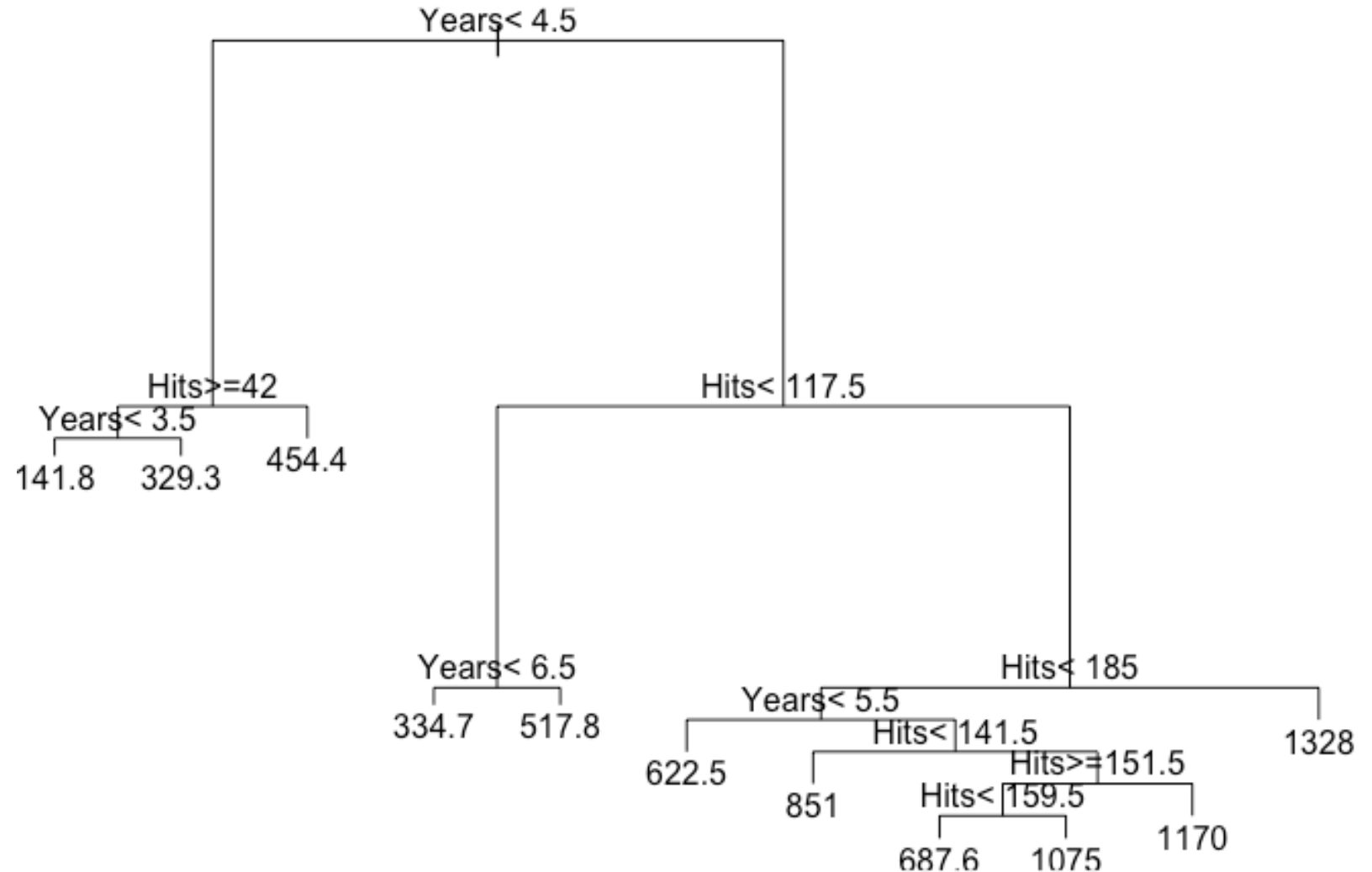
*optimum cut-point for *Hits*.

Recursive binary partitioning

- We first select the input variable x_j and the cut-point s such that **splitting the feature space** into the regions $\{x|x_j < s\}$ and $\{x|x_j \geq s\}$ leads to the **greatest possible reduction** in RSS.
- Next, we **repeat** the process, looking for the best input variable and best cut-point in order to split the data further so as to minimise the RSS within each of the resulting regions.
- However, this time, instead of splitting the entire predictor space, we split one of the two previously identified regions. We now have three regions.
- Again, we look to split one of these three regions further, so as to minimise the RSS. The process continues until a **stopping criterion is reached**; for instance, we may continue until no region contains more than five observations.

Recursive partitioning example – Hitters data (2 variables)

...but we can keep
splitting the space until
RSS converges



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$E(T, X) = \sum_{c \in X} P(c) E(c)$$

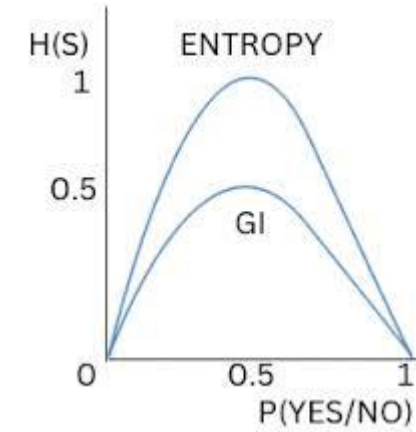
Play Golf	
Yes	No
9	5

$$\begin{aligned} \text{Entropy(PlayGolf)} &= \text{Entropy}(5,9) \\ &= \text{Entropy}(0.36, 0.64) \\ &= -(0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\ &= 0.94 \end{aligned}$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

$$\begin{aligned} E(\text{PlayGolf}, \text{Outlook}) &= P(\text{Sunny}) * E(3,2) + P(\text{Overcast}) * E(4,0) + P(\text{Rainy}) * E(2,3) \\ &= (5/14) * 0.971 + (4/14) * 0.0 + (5/14) * 0.971 \\ &= 0.693 \end{aligned}$$

Predictors				Target
Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
		Gain = 0.247	

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
		Gain = 0.029	

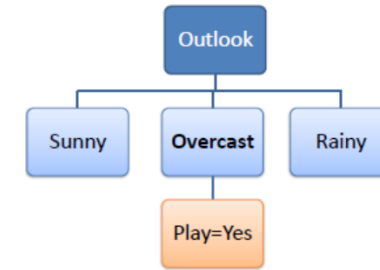
		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
		Gain = 0.152	

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
		Gain = 0.048	

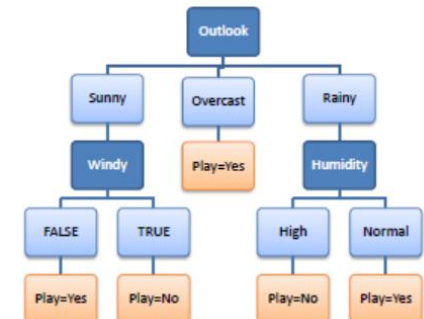
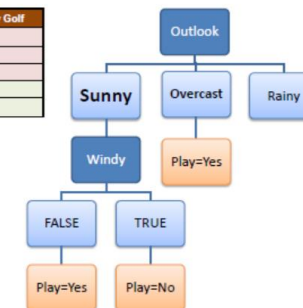
$$\text{Gain}(T, X) = \text{Entropy}(T) - \text{Entropy}(T, X)$$

$$\begin{aligned} G(\text{PlayGolf}, \text{Outlook}) &= E(\text{PlayGolf}) - E(\text{PlayGolf}, \text{Outlook}) \\ &= 0.940 - 0.693 = 0.247 \end{aligned}$$

Outlook	Temp	Humidity	Windy	Play Golf
Sunny	Mild	High	FALSE	Yes
	Cool	Normal	FALSE	Yes
	Cool	Normal	TRUE	No
	Mild	Normal	FALSE	Yes
	Mild	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
	Cool	Normal	TRUE	Yes
	Mild	High	TRUE	Yes
	Hot	Normal	FALSE	Yes
Rainy	Hot	High	FALSE	No
	Hot	High	TRUE	No
	Mild	High	FALSE	No
	Cool	Normal	FALSE	Yes
	Mild	Normal	TRUE	Yes

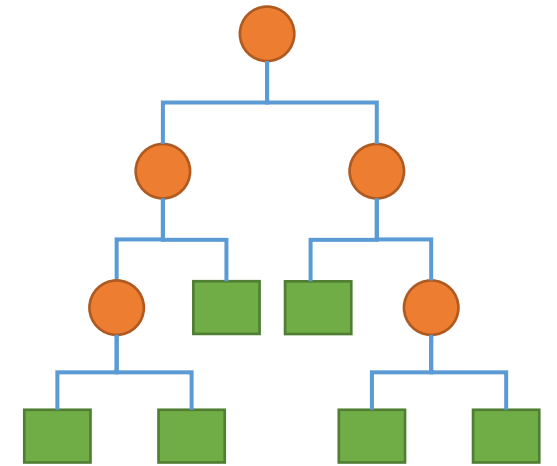
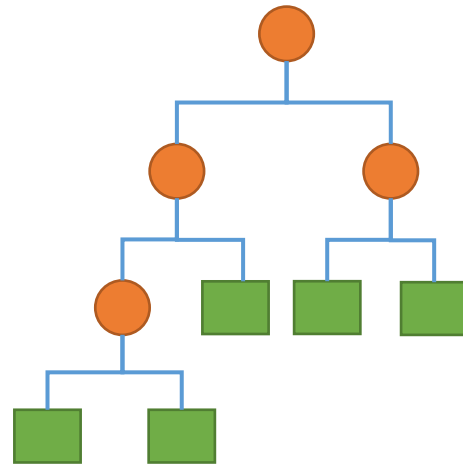
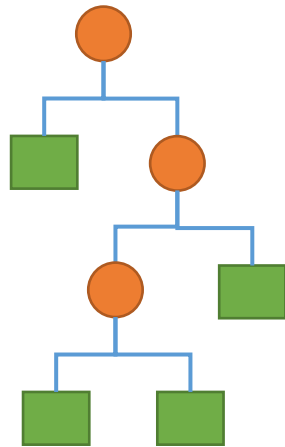
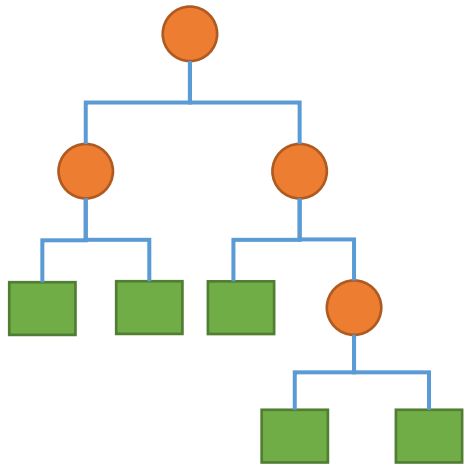


Temp	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



Issues with Decision Trees

- Generally, Decision Trees don't perform well.
- The same data may be partitioned **differently**, leading to different decision tree models:

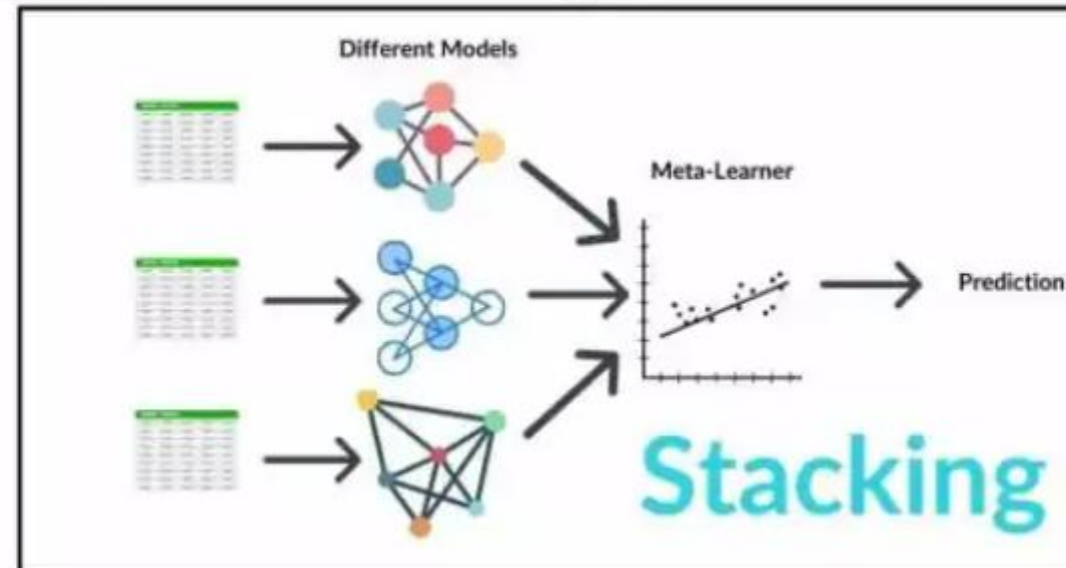
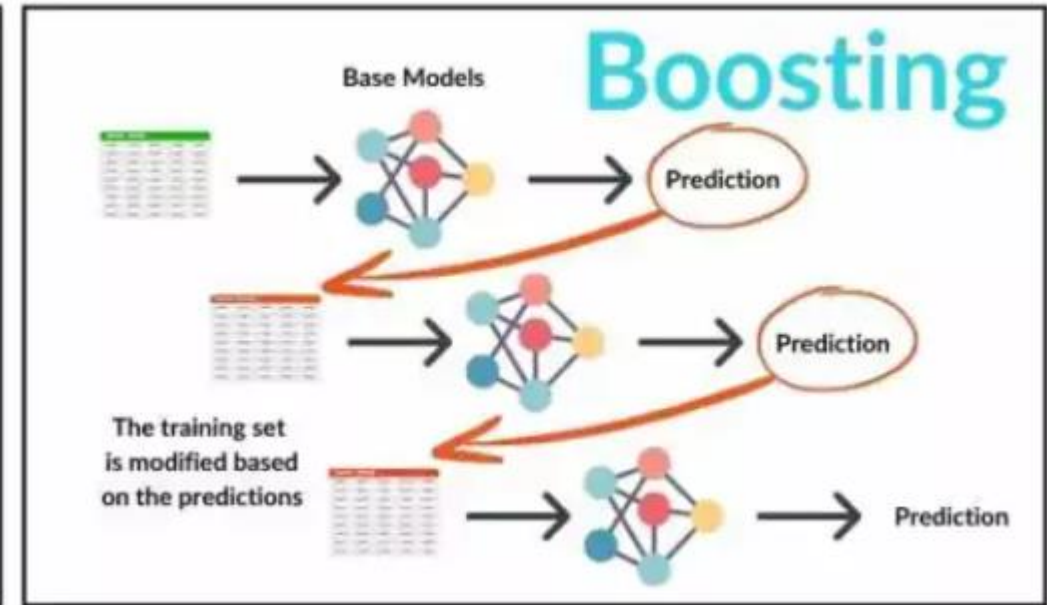
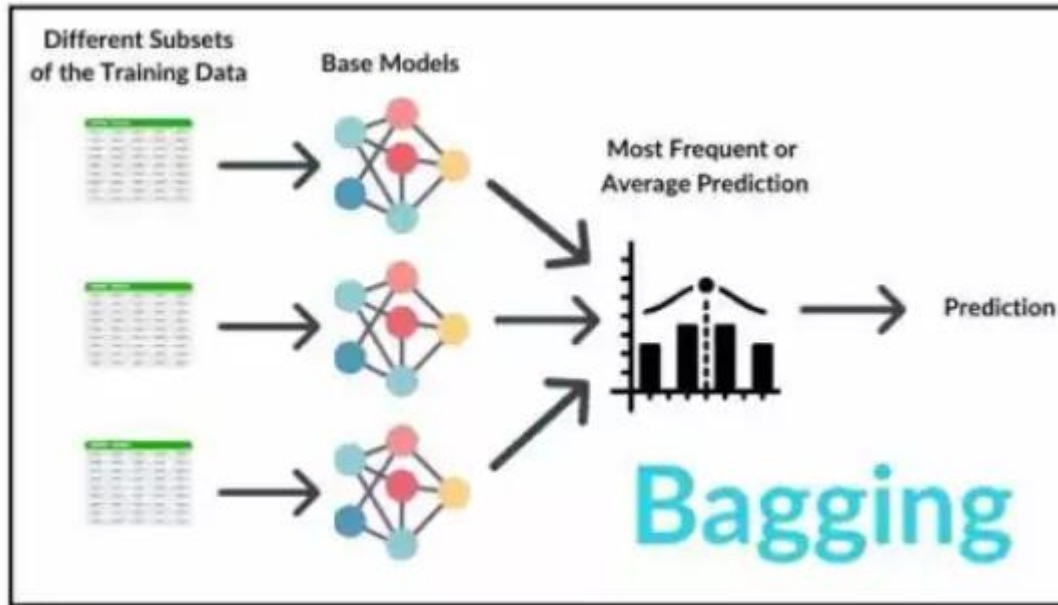


Pruning

- A better strategy is to grow a **very large** tree T_0 , and then **prune** it back in order to obtain a subtree.
- Cost complexity pruning:

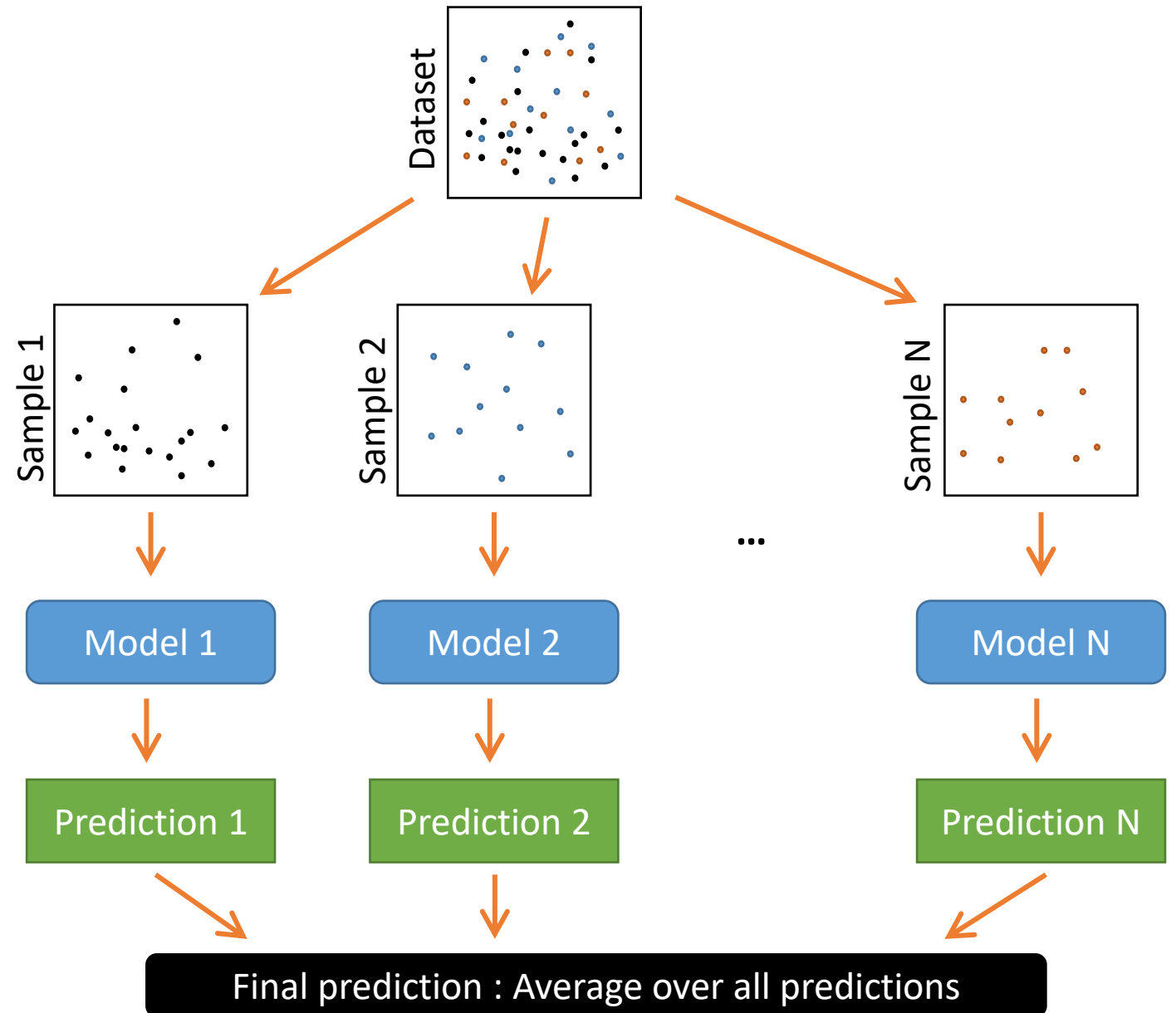
$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

- α - complexity hyperparameter: It is a nonnegative tuning parameter that controls the trade-off between the subtree's complexity and its fit to the training data.
- For each value of α there corresponds a subtree $T \subset T_0$.
- $|T|$ - indicates the number of terminal nodes of the tree T .
- α can be selected by cross-validation.



Bagging

- Bagging is a ML technique that takes **repeated samples** from the training set and build a model for each of them.
- The final prediction is either the average (regression) or the mode (classification) of the model predictions.



Bagging

- Algorithm for regression:
 1. Generate B different bootstrapped training data sets.
 2. Train method on the b th bootstrapped training set in order to get $\hat{f}^{*b}(x)$, the prediction at a point x .
 3. The final prediction is the average over all predictions:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

- For classification – instead of taking the average, the overall predicted value (class) is the majority vote – i.e. the mode of all predictions:

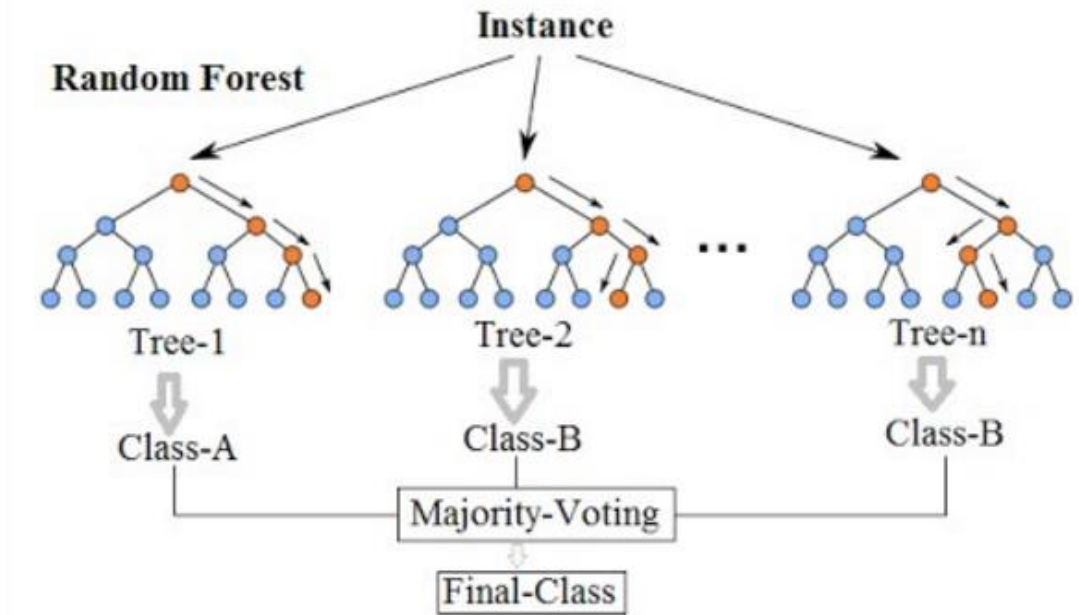
$$\hat{f}_{\text{bag}}(x) = \text{mode}_{b=1}^B \hat{f}^{*b}(x)$$

- In principle, bagging can be applied to **any** machine learning method. But it is usually used with decision trees.

Random forests

Random forests are **similar** to bagged trees, but with a tweak to decrease the variance even further:

- As in bagging, a number of decision trees are built on bootstrapped training samples.
- But when building these decision trees, each time a split in a tree is considered, a **random selection** of m input variables is chosen as split candidates from the full set of p .
- The split is allowed to use only one of those m predictors.
- Usually we choose $m \approx \sqrt{p}$



Variable importance measure

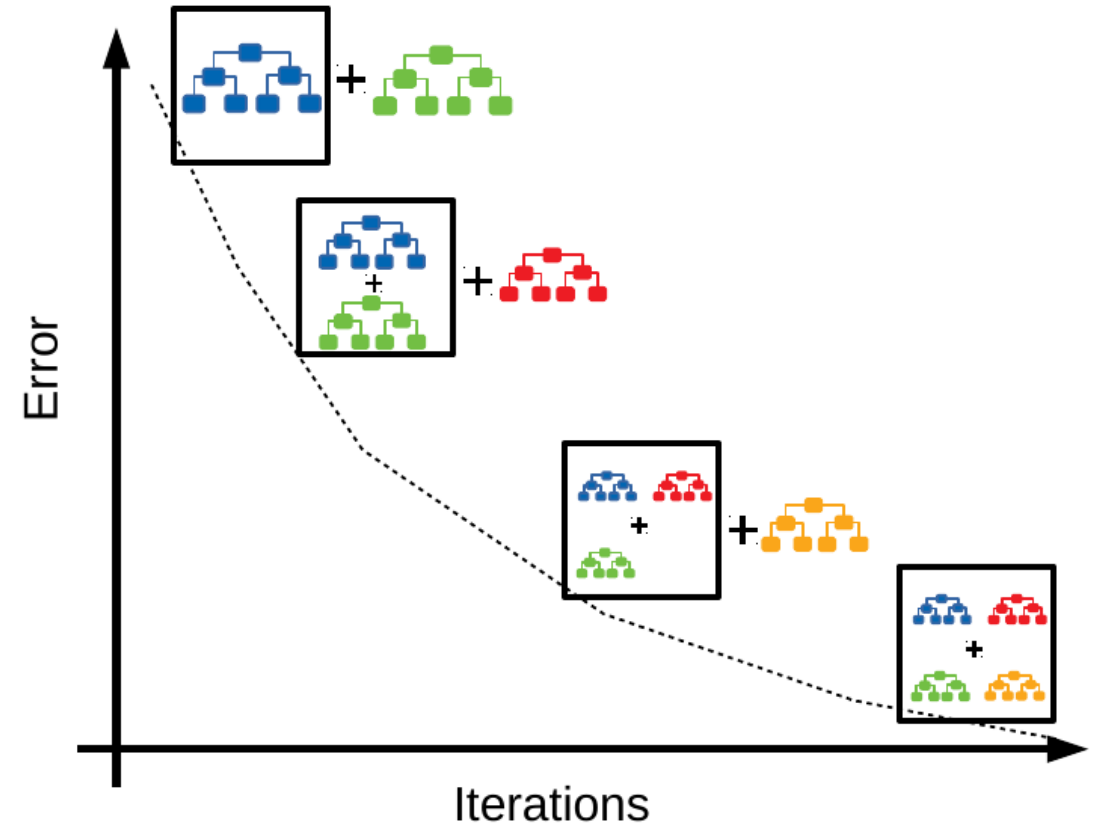
- One disadvantage of using bagged trees and RFs is the **lost of interpretability**, if we compare with decision trees.
- However, we can still measure variable importance:
- For regression:
 - We record the total amount that the RSS is decreased due to splits over a given input variable, averaged over all trees. A large value indicates an important variable.
- For classification:
 - We add up the total amount that the Gini index (or cross-entropy measure) is decreased by splits over a given input variable, averaged over all trees.

Boosting

- Let's assume a model F is built using a training set $\{\mathbf{X}, \mathbf{y} | (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ such that: $y = F(x) + \epsilon$.
 - The prediction error is: $\epsilon = y - F(x)$.
- Let's assume we want to improve the model by adding an additional one named h :
 - The new prediction will be $y = F(x) + h(x)$, or $h(x) = y - F(x)$.
- Therefore, in order to train $h(x)$ such that minimises the error, we can use the training set:
 - $\{\mathbf{X}, \mathbf{y} - \mathbf{F}(\mathbf{X}) | (x_1, y_1 - F(x_1)), (x_2, y_2 - F(x_2)), \dots, (x_n, y_n - F(x_n))\}$
- Note that $h(x) \rightarrow \epsilon$. Therefore $F(x) + h(x)$ should be closer to y than just $F(x)$.
 - That is, $F(x) + h(x)$ should be a better model than $F(x)$.
- Nothing stops us to now carry on adding another model after another.
- This modelling approach is called boosting.
- Models are consecutively added so the overall loss is minimised.

Gradient boosting machines (GBMs)

- **Common** ensemble techniques like random forests and bagging **rely on** simple **averaging** of models in the ensemble.
- Boosting methods is a family of ML/ensemble learning algorithms in which their main idea is to **add** new models to the ensemble **sequentially**.
- At each particular iteration, a new **weak**, base-learner model is trained with respect to the error of the whole ensemble learnt so far.
- Typically, decision trees are used as **base** learners.
- In gradient boosting machines, or simply, **GBMs**, the learning procedure **consecutively fits** new models to provide a more accurate estimate of the response variable.



Gradient boosting machines (GBMs)

- The principal idea behind this algorithm is to construct the new base-learners to be maximally correlated with the **negative gradient of the loss function**, associated with the whole ensemble.

$$y - F(x) \propto -\frac{\partial \mathcal{L}(F(x))}{\partial F(x)} = -\lambda \frac{\partial \mathcal{L}(F(x))}{\partial F(x)}$$

- Where λ is the learning rate or shrinkage factor.
- The **smaller** parameter λ and therefore, the lower the shrunk boosted increments are, the **better generalisation** is achieved at the **cost** of convergence speed.
- The loss functions applied can be **arbitrary**, but to give a better intuition, if the error function is the classic **squared-error loss**, the learning procedure would result in consecutive error-fitting.
- GBMs are highly customisable to any particular data-driven task.
- Boosting algorithms are relatively simple to implement, which allows one to experiment with different model designs.
- Moreover the GBMs have shown considerable **success** in not only practical applications, but also in various machine-learning and data-mining challenges

Types:

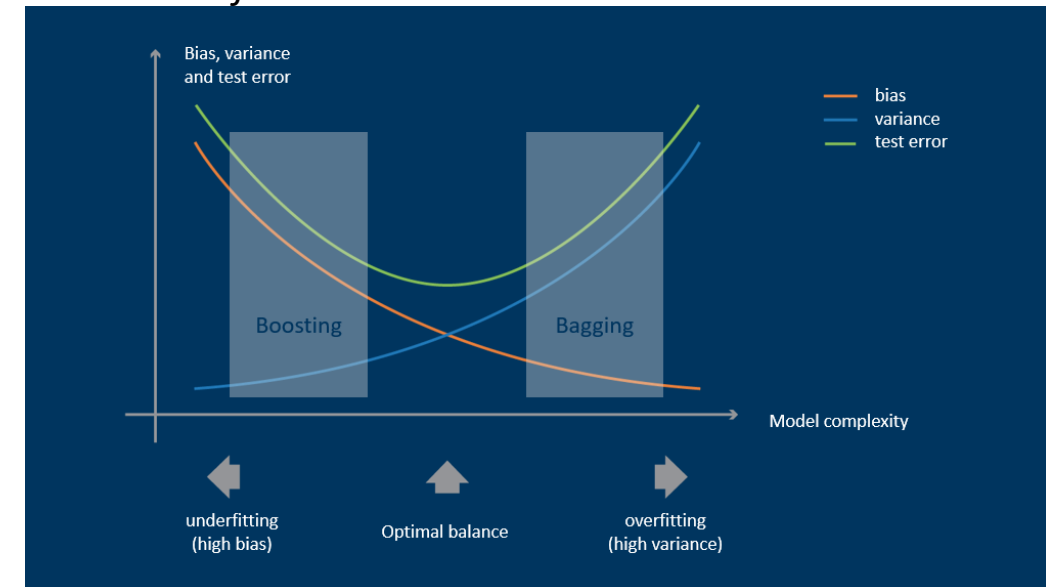
- AdaBoost (Adaptive Boosting)**: One of the first boosting algorithms. It assigns weights to instances and combines the weak learners' predictions through weighted majority voting.
- Gradient Boosting Machines (GBM)**: Builds an ensemble of models by iteratively fitting new models to the negative gradient of the loss function.
- XGBoost (Extreme Gradient Boosting)**: An optimized version of GBM that uses regularization, pruning, and efficient tree construction to improve performance.
- LightGBM**: A gradient-boosting framework that uses histogram-based techniques for faster training on large datasets.
- CatBoost**: A gradient boosting algorithm that handles categorical features efficiently and includes built-in methods to prevent overfitting.

AdaBoost: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html>

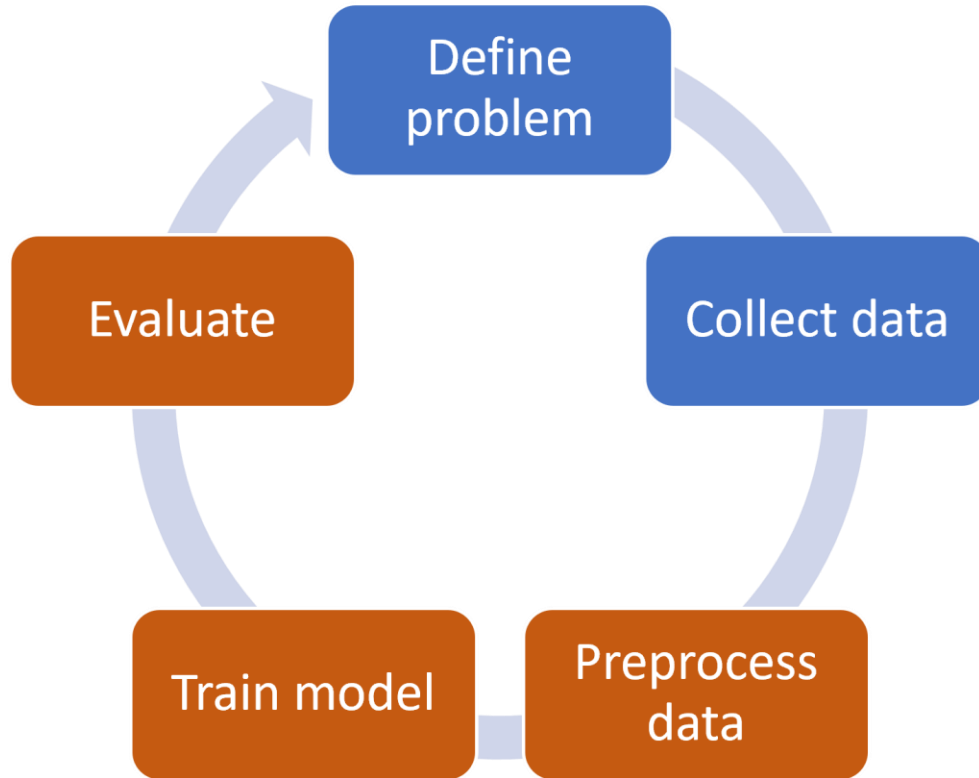
XGboost: https://xgboost.readthedocs.io/en/release_3.2.0/

Lightgbm: <https://lightgbm.readthedocs.io/en/stable/>

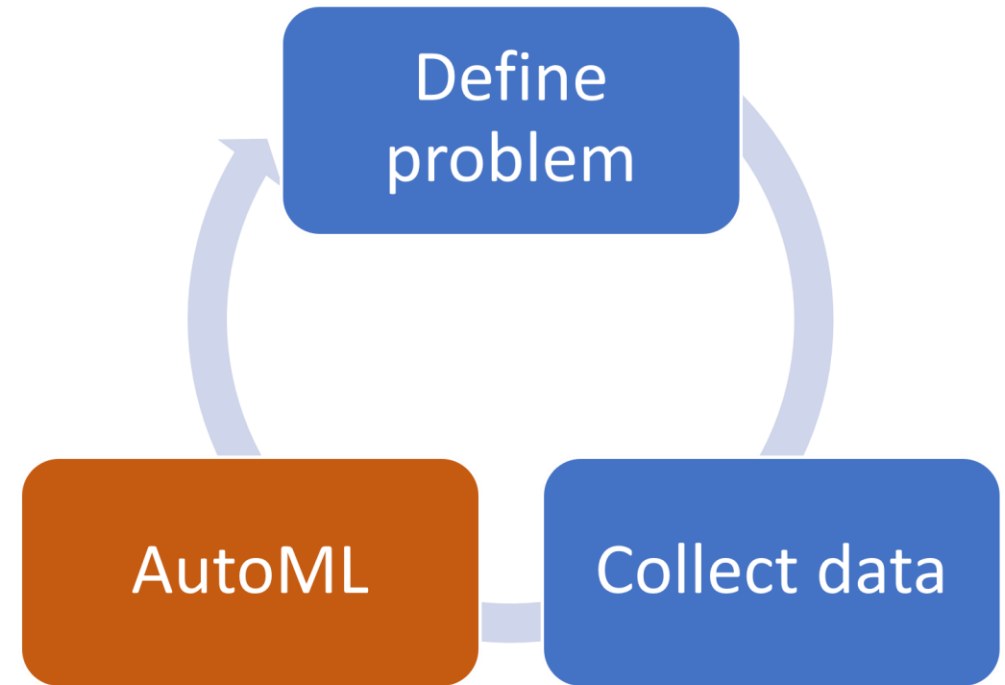
Catboost: <https://catboost.ai/>



Traditional ML training workflow



AutoML workflow



https://automl.github.io/auto-sklearn/master/examples/20_basic/example_classification.html#sphx-glr-examples-20-basic-example-classification-py

<https://pycaret.gitbook.io/docs> (2025-last update)

Discussion

- We studied representative supervised learning algorithms such as:
- Support Vector Machines
- Decision Trees
- Ensemble Methods
 - Random Forest
 - Gradient Boosting Machines